



VERIQTA

BUILD • AUTOMATE • DEPLOY • SCALE

DEVOPS
ENGINEER'S
HANDBOOK

Automate
Everything.
Deliver
Anything.

JENKINS

ADVANCED HANDBOOK

From Zero to Production

PRODUCTION
READY

- ✓ CI/CD
- ✓ Pipelines
- ✓ Security
- ✓ Scaling
- ✓ Monitoring
- ✓ Troubleshooting
- ✓ Best Practices

BUILD
↓
TEST
↓
DEPLOY



DEVELOPER



GITHUB



JENKINS



BUILD



TEST



PACKAGE



DOCKER



KUBERNETES

COVERING

20
PAGES

of In-Depth
Jenkins Notes



SECURE
HARDENED
JENKINS



REUSABLE
SHARED LIBRARIES
& PIPELINES



MONITOR
METRICS, ALERTS
& DASHBOARDS



PERFORM
OPTIMIZE FOR
SPEED & SCALE



PRODUCTION
READY CHECKLISTS
& BEST PRACTICES

WHAT YOU WILL MASTER



REAL CASES
Production Lessons



TROUBLESHOOT
Like a Pro



INTERVIEW
Q&A Included



CHECKLISTS
For Success



CLOUD AGNOSTIC
Anywhere, Anytime



THE COMPLETE JENKINS REFERENCE

TO DESIGN, BUILD, AUTOMATE & DEPLOY AT SCALE



Learn it.
Build it. ✓
Automate it. ✓
Own it.



1. JENKINS OVERVIEW

1. WHAT IS JENKINS?

Jenkins is an open-source automation server that helps developers build, test and deploy their code reliably and continuously.

It is the most popular CI/CD tool used worldwide.



Jenkins is **NOT** just a build tool. It is an automation platform for the entire software delivery lifecycle.

2. WHY JENKINS EXISTS

- Automate repetitive tasks.
- Integrate code changes frequently.
- Detect issues early through automated builds & tests.
- Deliver software faster and more reliably.
- Provide visibility into the delivery pipeline.
- Enable collaboration between Dev and Ops.

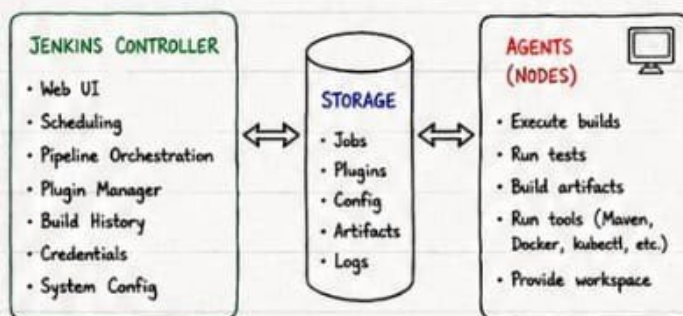
3. CI vs CD vs CONTINUOUS DEPLOYMENT

CI (Continuous Integration) : Developers integrate code frequently. Builds & tests run automatically.

CD (Continuous Delivery) : Code is always in a deployable state. Deployment to staging is automated.

Continuous Deployment : Every change that passes all stages is automatically deployed to production.

4. JENKINS ARCHITECTURE (HIGH LEVEL)



Controller manages the system. Agents do the heavy lifting.

5. CONTROLLER vs AGENT

Controller (Master)

- Brain of Jenkins.
- Manages jobs, schedules builds.
- Handles pipeline execution flow.
- Stores configurations, credentials, plugins, histories.

Agent (Node/Slave)

- Executes build steps.
- Runs tests, builds code, creates artifacts.
- Can be physical, virtual or containers.
- Scales horizontally.

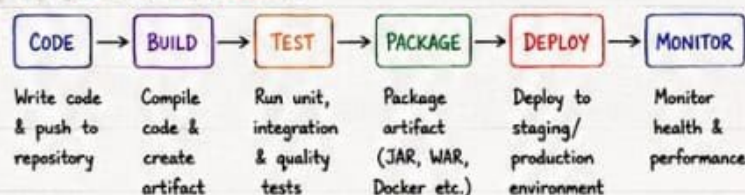
Think:
Controller
Plans,
Agent
Executes

6. WHY COMPANIES USE JENKINS

- ✓ Open Source & Free
- ✓ Huge Ecosystem (1800+ plugins)
- ✓ Highly Extensible
- ✓ Strong Community Support
- ✓ Works on Any Platform (Linux, Windows, macOS)
- ✓ Scales with Agents
- ✓ Integrates with Everything (Git, Docker, Kubernetes, AWS, etc.)
- ✓ Enterprise Proven & Battle Tested

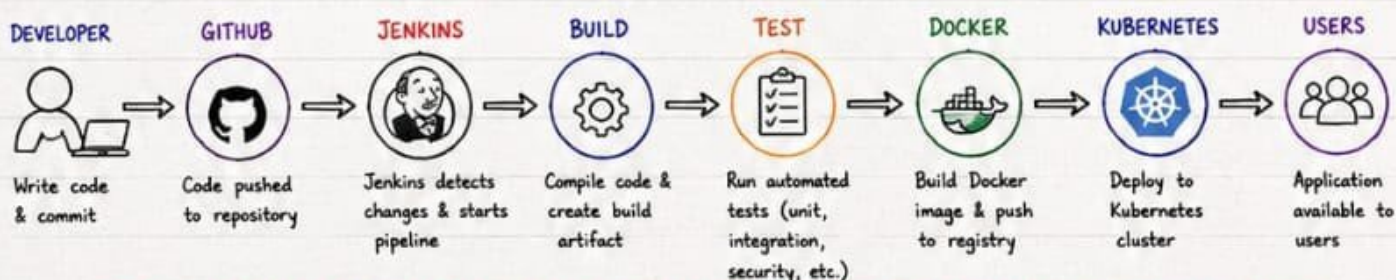
Trusted by thousands of companies including:
Netflix
Spotify
Uber
LinkedIn
Adobe
and many more!

7. PIPELINE LIFECYCLE



Automation at every step = Faster, Reliable, Repeatable

8. REAL PRODUCTION WORKFLOW (END TO END)



Goal: Deliver value to users continuously with **high quality** and **reliability**.



2. JENKINS ARCHITECTURE

1. OVERVIEW

Jenkins follows a Master-Agent (Controller-Agent) architecture. The Controller manages the system while Agents do the heavy lifting (builds, tests, deployments, etc).



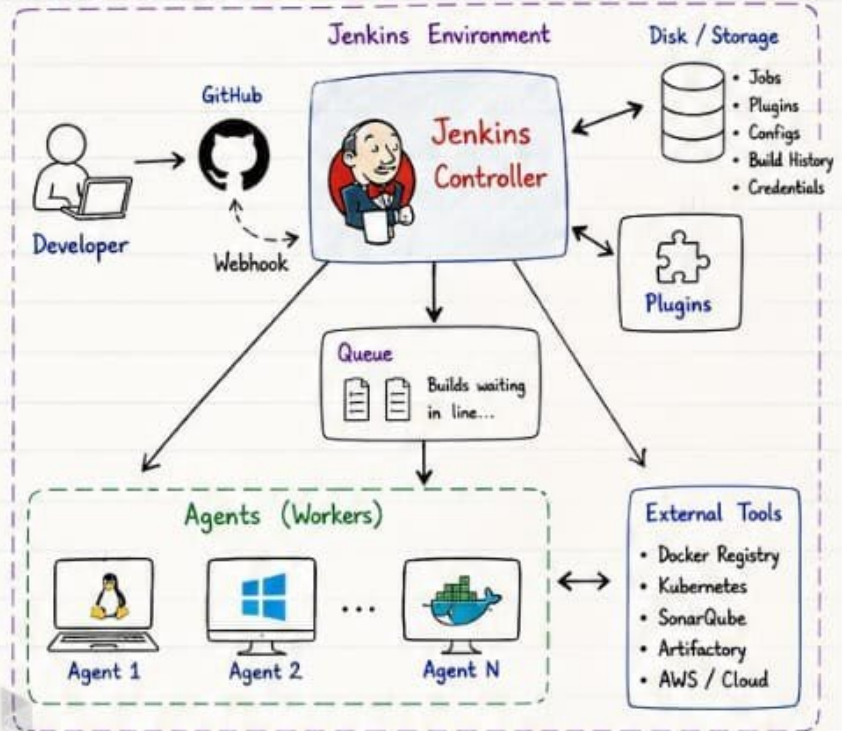
Key Idea:

Controller = Brain
Agents = Muscle

3. COMPONENTS EXPLAINED

- ① **Controller (Master)**
Runs Jenkins core. Manages jobs, schedules builds, stores configs, plugins, credentials and build history.
- ② **Agents (Workers)**
Execute the actual builds & tests. Can be static (permanent) or dynamic (created on demand e.g. Kubernetes, EC2).
- ③ **Queue**
Stores build requests when all executors are busy.
- ④ **Executors**
Slots on the Controller or Agent that run builds. Responsible for running one build at a time.
- ⑤ **Plugins**
Extend Jenkins functionality. Jenkins has 1800+ plugins.
- ⑥ **Storage**
Stores all persistent data like jobs, configs, plugins, logs, artifacts, etc.

2. ARCHITECTURE DIAGRAM



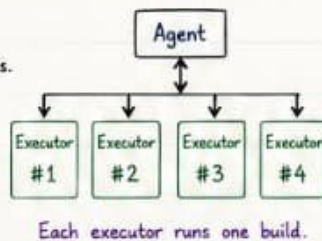
4. CONTROLLER vs AGENT COMPARISON

	CONTROLLER (MASTER)	AGENT (WORKER)
Role	Manages the system, coordinates builds, scheduling, and stores data.	Executes the actual build steps and tasks.
Responsibilities	- Manage jobs & pipelines - Schedule builds - Provide web UI - Manage plugins, credentials, configs - Track build history	- Run builds - Run tests - Create artifacts - Can be offline without affecting controller
Where runs?	Usually on a dedicated server / VM	On multiple machines (VM, Docker, K8s, Cloud, Bare-metal)
Resource usage	Medium (depends on executor count)	High (CPU, RAM, Disk used for builds)
Can it build?	Yes, but not recommended for heavy builds.	Yes! Recommended.
Failure impact	If Controller fails → whole Jenkins is down.	If Agent fails → only that agent stops building.

5. EXECUTORS

Executors are the slots that run builds.

- Controller can have executors
- Agents have executors
- One executor = One build at a time



6. DISTRIBUTED BUILDS

Jenkins supports distributed builds.

Agents can be located:

- On-prem servers
- Cloud (AWS, Azure, GCP)
- Docker containers
- Kubernetes pods (dynamic)

→ Builds run closer to the code, faster and in parallel!

7. INTERVIEW NOTES

- ★ Why Master-Agent? → Scalability, reliability, isolation and performance.
- ★ Can Jenkins controller and agent be on same machine? → Yes, but not recommended for production.
- ★ What happens if agent goes offline? → Builds on that agent stop, others continue.
- ★ Can multiple agents run same job? → Yes, Jenkins schedules builds on any available executor.

Remember!

- ✓ Think Controller = Manage
- ✓ Think Agent = Execute
- ✓ Think Executors = Capacity
- ✓ Think Queue = Waiting Line
- ✓ Think Plugins = Power-ups



3. INSTALLING JENKINS

Jenkins can be installed in multiple ways. The two most common methods are Docker (recommended) and Linux (package installation).

1. DOCKER INSTALLATION (RECOMMENDED)

```
# Pull Jenkins image
docker pull jenkins/jenkins:lts

# Create a network
docker network create jenkins

# Run Jenkins container
docker run -d --name jenkins \
--restart unless-stopped \
-p 8080:8080 -p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
--network jenkins jenkins/jenkins:lts
```

Ports

- 8080 → Web UI
- 50000 → Agent Communication

Volumes

jenkins_home
→ Stores all Jenkins data persistently

Useful Docker Commands

```
docker ps -a          # List containers
docker logs -f jenkins # View Jenkins logs
docker restart jenkins # Restart container
docker exec -it jenkins bash # Access container
docker stop jenkins    # Stop Jenkins
```

2. LINUX INSTALLATION (UBUNTU)

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Java (Jenkins requires Java 17+)
sudo apt install openjdk-17-jdk -y

# Add Jenkins repository key
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key \
| sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

# Add Jenkins repository
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | \
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# Install Jenkins
sudo apt update
sudo apt install jenkins -y

# Start Jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins

# Check Status
sudo systemctl status jenkins
```



Jenkins runs as:
User: jenkins
Service: jenkins
Port: 8080

3. INITIAL ADMIN PASSWORD

After starting Jenkins, get the initial admin password:

```
# Docker
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

# Linux
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```



4. UNLOCK JENKINS

- 1 Open your browser → <http://localhost:8080>
- 2 Jenkins will ask for the initial admin password
- 3 Paste the password → Click Continue
- 4 Choose "Install suggested plugins"
- 5 Wait for installation to complete

Jenkins is unlocked and ready! 🎉



5. INSTALL PLUGINS (ESSENTIAL)

- ✓ Git - Git integration
- ✓ Pipeline - For CI/CD pipelines
- ✓ Docker Pipeline - Build Docker images
- ✓ Kubernetes - Deploy to Kubernetes
- ✓ Credentials - Manage secrets
- ✓ Blue Ocean - Beautiful pipeline UI
- ✓ Email Extension - Notifications
- ✓ OWASP Dependency Check - Security scanning

Tip

Install only what you need.
Too many plugins = slow & unstable Jenkins.



6. CREATE FIRST ADMIN USER

- 1 Create First Admin User
 - Username
 - Password
 - Full Name
 - Email
- 2 Click Save and Finish
- 3 Jenkins is ready to use!

Create First Admin User

Username:
Password:
Full Name:
Email:

Save and Finish

7. BEST PRACTICES

- ★ Run Jenkins behind reverse proxy (Nginx)
- ★ Use HTTPS (SSL/TLS)
- ★ Do not run Jenkins as root
- ★ Regular backups of \$JENKINS_HOME
- ★ Limit plugin installations
- ★ Keep Jenkins & plugins updated
- ★ Manage credentials securely
- ★ Use dedicated agents for builds



8. IMPORTANT COMMANDS (LINUX)

```
sudo systemctl start jenkins # Start Jenkins
sudo systemctl stop jenkins  # Stop Jenkins
sudo systemctl restart jenkins # Restart Jenkins
sudo systemctl status jenkins # Check Status
sudo journalctl -u jenkins -f # View Logs
sudo cat /var/lib/jenkins/secrets/initialAdminPassword # Admin Password
sudo tail -f /var/log/jenkins/jenkins.log # Logs
```

9. TROUBLESHOOTING

- ⚠ Jenkins not starting?
→ Check logs: `journalctl -u jenkins -f`
- ⚠ Port 8080 in use?
→ Change port or stop other service
- ⚠ Can't access Jenkins?
→ Check firewall / security groups
- ⚠ Plugins fail to install?
→ Check internet / proxy settings
- ⚠ Out of memory?
→ Increase Java heap size (See below)

Increase JVM Memory

Edit file:
`/etc/default/jenkins`
Set:
`JAVA_OPTS="`
`-Xms512m`
`-Xmx2048m`

Restart Jenkins

QUICK REFERENCE FLOW



NOTE

This is just the beginning.
Next up → Jenkins Jobs & Pipelines! ☆





4. JENKINS JOBS

Jenkins provides different job types to handle various use cases.
Choose the right job for the right problem!

Remember:

- Jobs = Automations
- Pipelines = Workflows
- Build = Execution



1. TYPES OF JENKINS JOBS



FREESTYLE PROJECT

- Traditional job type
- Task based
- Configured via UI
- Build steps
- Post build actions

Use when:

Simple tasks, quick automation, no complex flow.



PIPELINE (DECLARATIVE)

- Jenkinsfile (Declarative Syntax)
- Structured & opinionated
- Easy to read
- Best for standard CI/CD

Use when:

You want a standard CI/CD pipeline with best practices.



PIPELINE (SCRIPTED)

- Jenkinsfile (Groovy)
- More flexible
- Full control
- Programmatic

Use when:

You need maximum flexibility and custom logic.



MULTIBRANCH PIPELINE

- Automatically scans repo
- Creates pipeline for each branch
- Great for teams
- PR & branch builds

Use when:

You have multiple branches/PRs and want automation.



ORGANIZATION FOLDER

- Manage many jobs/folders
- RBAC, credentials
- Shared libraries
- Governance

Use when:

Enterprise setup with many teams & projects.



MATRIX PROJECT

- Test combinations
- OS, JDK, Browser etc.
- Parallel across dimensions

Use when: You need to test multiple combinations.



FOLDER

- Organize jobs
- No build capability
- Simple grouping

Use when: You want to group jobs.



MULTICONFIGURATION

- Parameterized builds
- Different configs
- Same job, many variants

Use when: You need multiple build configs.



EXTERNAL JOB

- Triggers job on remote Jenkins
- No local execution

Use when: You want to trigger jobs on another Jenkins instance.

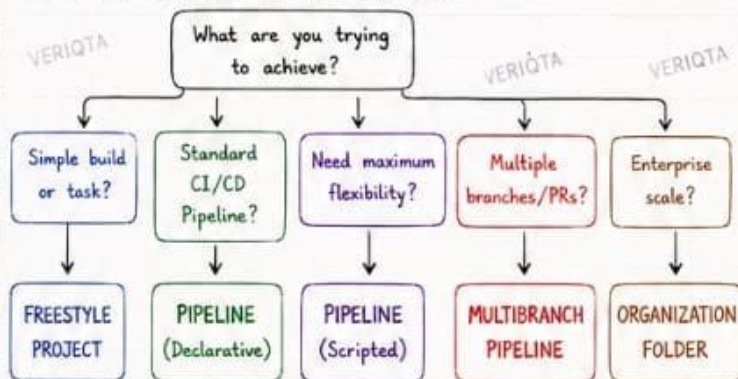


VIEW

- List of jobs
- Filter, group, and monitor

Use when: You want visibility & tracking.

2. WHICH JOB SHOULD YOU CHOOSE?



★ Tip: Start simple. Move to advanced as your needs grow.

3. REAL-WORLD EXAMPLES

SCENARIO	BEST JOB TYPE
Build Java App and run tests	Pipeline (Declarative)
Build all branches & PRs from GitHub	Multibranch Pipeline
Test on multiple OS & JDK combinations	Matrix Project
Custom complex deployment logic	Pipeline (Scripted)
Manage 100+ teams and projects	Organization Folder
Trigger job on another Jenkins server	External Job

4. USEFUL CLI COMMANDS (JENKINS CLI)

```

java -jar jenkins-cli.jar -s http://localhost:8080/ help # List all commands
java -jar jenkins-cli.jar -s http://localhost:8080/ list-jobs # List all jobs
java -jar jenkins-cli.jar -s http://localhost:8080/ build JOB_NAME # Trigger build
java -jar jenkins-cli.jar -s http://localhost:8080/ stop-build JOB_NAME 5 # Stop build (build #5)
java -jar jenkins-cli.jar -s http://localhost:8080/ disable-job JOB_NAME # Disable job
java -jar jenkins-cli.jar -s http://localhost:8080/ enable-job JOB_NAME # Enable job
java -jar jenkins-cli.jar -s http://localhost:8080/ who-am-i # Current user
java -jar jenkins-cli.jar -s http://localhost:8080/ safe-restart # Restart Jenkins
  
```

5. BEST PRACTICES

- ✓ Use Pipeline (Declarative) for 95% of use cases.
- ✓ Store Jenkinsfiles in Git.
- ✓ Use Multibranch Pipeline for modern workflows.
- ✓ Keep jobs small and focused.
- ✓ Use folders and Organization Folders to organize.
- ✓ Use credentials store, NEVER hardcode secrets.
- ✓ Enable build discarder to save disk space.
- ✓ Use shared libraries for reusable code.



Clean Jobs
= Happy Pipelines!

★ INTERVIEW NOTE

Q: Which job type will you use to build a microservice project with multiple branches?

A: Multibranch Pipeline. It automatically creates pipelines for each branch and PR.

Right Job Choice
= Efficient Automation





5. JENKINS PIPELINE

Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins.



Key Idea:

Pipeline as Code → Versioned, Reusable, Reliable and Automated end-to-end.

1. DECLARATIVE PIPELINE

- Structured and opinionated.
- Easier to write and read.
- Designed for most common use cases.
- Uses a fixed syntax and blocks.



Use when:

You want a standard CI/CD pipeline with best practices.

2. SCRIPTED PIPELINE



- More flexible and powerful.
- Uses Groovy language.
- No fixed structure.
- Better for complex logic.



Use when:

You need maximum flexibility, custom logic, loops, conditions, dynamic behavior.

Which one?

- Start with Declarative.
- Switch to Scripted when you hit limits.



3. PIPELINE SYNTAX – KEY SECTIONS

`pipeline { }`

- The root block that defines a pipeline.

```
pipeline {
  ...
}
```

`agent { }`

- Defines where the pipeline or stage runs.

```
agent any
agent { label 'linux' }
agent { docker 'maven:3.9' }
```

`stages { }`

- Contains all the stages of your pipeline.

```
stages {
  stage('Build') { ... }
  stage('Test') { ... }
}
```

`stage('Name') { }`

- Logical step in the pipeline.
- Groups related steps.

```
stage('Build') {
  steps { ... }
}
```

`steps { }`

- List of tasks executed inside a stage.

```
steps {
  sh 'mvn clean install'
  echo 'Done'
}
```

`environment { }`

- Set environment variables globally or per stage.

```
environment {
  ENV = 'production'
  APP = 'myapp'
}
```

`parameters { }`

- Define inputs to make pipeline dynamic.

```
parameters {
  string(name: 'ENV',
    defaultValue: 'dev')
}
```

`post { }`

- Actions to run after pipeline or stage (always, success, failure, etc).

```
post {
  always { echo 'Done' }
  failure { mail ... }
}
```

`options { }`

- Control pipeline behavior.

```
options {
  timeout(time: 30, unit: 'MINUTES')
  timestamps()
  buildDiscarder(...)
}
```

`triggers { }`

- Define how/when the pipeline should start.

```
triggers {
  pollSCM('* * * * *')
}
```

4. EXAMPLE PIPELINES

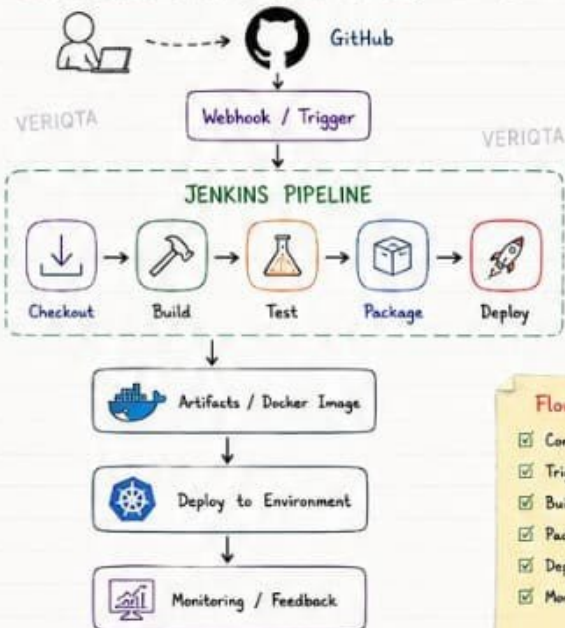
A. DECLARATIVE PIPELINE

```
pipeline {
  agent any
  environment {
    ENV = 'dev'
  }
  stages {
    stage('Build') {
      steps {
        echo 'Building the application...'
        sh 'mvn clean package'
      }
    }
    stage('Test') {
      steps {
        echo 'Running tests...'
        sh 'mvn test'
      }
    }
  }
  post {
    always { echo 'Pipeline completed' }
    failure { echo 'Build failed!' }
  }
}
```

B. SCRIPTED PIPELINE

```
node('linux') {
  stage('Checkout') {
    git 'https://github.com/user/repo.git'
  }
  stage('Build') {
    sh 'mvn clean package'
  }
  stage('Test') {
    try {
      sh 'mvn test'
    } catch (err) {
      currentBuild.result = 'FAILURE'
      throw err
    }
  }
  stage('Deploy') {
    sh './deploy.sh'
  }
}
```

5. PIPELINE DIAGRAM (HIGH LEVEL)



Flow:

- ☒ Commit Code
- ☒ Trigger Pipeline
- ☒ Build & Test
- ☒ Package Artifact
- ☒ Deploy
- ☒ Monitor & Feedback



Remember!

Pipeline = Code
Version it, review it,
and improve it.



WHY PIPELINE AS CODE?

- ☒ Version Control
- ☒ Review & Approval
- ☒ Reusability
- ☒ Audit & Compliance
- ☒ Visibility
- ☒ Faster Delivery



TIP

Keep your Jenkinsfile clean,
modular and well-documented.
Small steps. Clear stages.
Happy Pipelines! 😊





6. JENKINSFILE DEEP DIVE

A Jenkinsfile defines your entire CI/CD pipeline as Code.



KEY IDEA

Infrastructure as Code
+
Pipeline as Code
=
Automated Everything!

1. COMPLETE JENKINSFILE ANATOMY

```

1 pipeline {
2   agent any
3   options {
4     timestamps()
5     buildDiscarder(logRotator(numToKeepStr: '20'))
6   }
7   environment {
8     APP_NAME = 'myapp'
9     IMAGE_TAG = "${env.BUILD_NUMBER}"
10    DOCKER_REGISTRY = 'docker.io/myorg'
11  }
12  parameters {
13    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Deploy Environment')
14  }
15  stages {
16    stage('Build') {
17      steps {
18        echo "🔨 Building the application..."
19        sh 'mvn clean package -DskipTests'
20      }
21    }
22    stage('Test') {
23      steps {
24        echo "🧪 Running unit tests..."
25        sh 'mvn test'
26      }
27      post {
28        always { junit 'target/surefire-reports/*.xml' }
29      }
30    }
31    stage('Docker') {
32      steps {
33        echo "🐳 Building Docker image..."
34        sh 'docker build -t ${DOCKER_REGISTRY}/${APP_NAME}:${IMAGE_TAG}'
35        sh 'docker push ${DOCKER_REGISTRY}/${APP_NAME}:${IMAGE_TAG}'
36      }
37    }
38    stage('Deploy') {
39      steps {
40        echo "🚀 Deploying to ${params.ENV} ..."
41        sh 'kubectl apply -f k8s/${params.ENV}/'
42      }
43    }
44  }
45  post {
46    success {
47      echo '✅ Pipeline succeeded!'
48      slackSend channel: '#deployments', color: 'good', message: "🎉 ${env.JOB_NAME} #${env.BUILD_NUMBER} Success"
49    }
50    failure {
51      echo '❌ Pipeline failed!'
52      slackSend channel: '#deployments', color: 'danger', message: "💥 ${env.JOB_NAME} #${env.BUILD_NUMBER} Failure"
53    }
54    always {
55      cleanWs()
56    }
57  }
58 }

```

Pipeline Block
Defines the pipeline.
Top-level block.

Agent
Where the pipeline
will run (any / label
/ docker / node).

Options
✔ Timestamps in logs
✔ Keep only last 20 builds

Environment Variables
Global variables available
throughout the pipeline.

Parameters
Allows user input
when running the job.

Stages

- 🔨 **Build** → Compile the code
- 🧪 **Test** → Run unit tests
- 🐳 **Docker** → Build & Push image
- 🚀 **Deploy** → Deploy to K8s

Post Actions

- Runs after the pipeline/stages.
- ✔ success → on success
 - ✔ failure → on failure
 - ✔ always → always run

BENEFITS

- ✔ Version controlled
- ✔ Reusable
- ✔ Consistent
- ✔ Reviewable
- ✔ Can run anywhere
- ✔ Audit friendly

2. STAGE BREAKDOWN

	BUILD STAGE	- Checkout code (implicit in SCM trigger or explicit) - Compile the application - Package artifacts (JAR/WAR/ZIP)
	TEST STAGE	- Run unit tests - Code quality checks (SonarQube, Lint) - Generate test reports (JUnit, TestNG)
	DOCKER STAGE	- Build Docker image - Tag image with build number / commit - Push image to registry (DockerHub/ECR/GCR)
	DEPLOY STAGE	- Deploy to target environment - Update Kubernetes manifests - Rollout / Check status
	NOTIFICATIONS	- Success / Failure alerts - Slack / Email / Teams / Discord - Inform the right people

Think Pipeline,
Think Automation,
Think Reliability!

3. NOTIFICATIONS EXAMPLES

SLACK EXAMPLE

```

slackSend(
  channel: '#deployments',
  color: 'good',
  message: "✅ ${env.JOB_NAME} #${env.BUILD_NUMBER} Success"
)

```

EMAIL EXAMPLE

```

emailSend(
  subject: "Jenkins Build ${currentBuild.result}",
  body: "Build #${env.BUILD_NUMBER} is ${currentBuild.result}",
  to: 'team@example.com'
)

```

TEAMS EXAMPLE (Webhook)

```

httpRequest(
  url: 'https://outlook.office.com/webhook/...',
  httpMode: 'POST',
  requestBody: "{ 'text': 'Build ${env.BUILD_NUMBER} ${currentBuild.result}' }"
)

```

4. BEST PRACTICES

- ✔ Keep Jenkinsfile clean and modular.
- ✔ Use environment variables.
- ✔ Use parameters for flexibility.
- ✔ Keep stages small and focused.
- ✔ Use try/catch for critical steps.
- ✔ Always clean workspace.
- ✔ Do not hardcode credentials.
- ✔ Use shared libraries for reusability.
- ✔ Add post always block.
- ✔ Version control everything.

5. QUICK TIPS

- ★ Use sh "set -e" for fail-fast scripts.
- ★ Use parallel {} for faster builds.
- ★ Use input step for manual approval.
- ★ Use when {} for conditional execution.
- ★ Keep logs readable and meaningful.



Jenkins Pipeline = Code + Automation + Reliability





7. GIT INTEGRATION

Jenkins integrates with Git to automatically detect changes, trigger builds, and manage code from development to production.

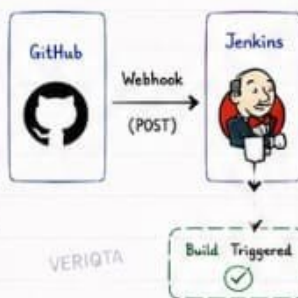
1. GITHUB WEBHOOKS



- GitHub sends an HTTP POST request to Jenkins when events happen.
- No polling → Instant builds!
- Events: push, pull_request, merge, tag, release, etc.

HOW IT WORKS

1. Code pushed to GitHub
2. GitHub triggers Webhook
3. Jenkins receives the event
4. Jenkins starts the build



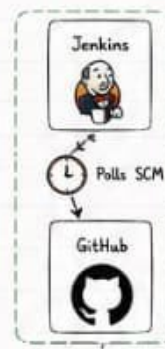
2. POLL SCM (NO WEBHOOKS?)



- Jenkins periodically checks the repository for changes.
- Simpler setup but not instant.
- Uses cron syntax.

EXAMPLE CRON SYNTAX

* * * * * → Every minute
 H/5 * * * * * → Every 5 minutes
 H/15 * * * * * → Every 15 minutes
 @hourly → Every hour



WEBHOOK vs POLL SCM

Webhook	Poll SCM
☒ Instant	☒ Simple
☒ Efficient	☒ Uses more resources
☒ Real-time	☒ Delay in build
☒ Recommended	

3. BRANCHES



- Build specific branches
- Use wildcards for multiple branches
- Example: */main, */develop, release/*

BRANCH STRATEGIES

- ☒ Main Branch → Production builds
- ☒ Develop Branch → Integration builds
- ☒ Feature Branches → Feature validation
- ☒ Release Branches → Pre-release builds
- ☒ Tag Builds → Versioned releases

EXAMPLE REPOSITORY



4. PULL REQUEST (PR) BUILDS



- Build code from Pull Requests before merging.
- Catch issues early.
- Works with GitHub PR events.



USE CASES

- ☒ Code review
- ☒ Automated tests
- ☒ Quality checks
- ☒ Comment build results on PR

Tip

Use the "GitHub Pull Request Builder" plugin or Multibranch Pipeline for best PR experience.

5. MERGE BUILDS



- Trigger builds when PR is merged.
- Ensures only merged code is built and deployed.



6. GIT CREDENTIALS



- Jenkins needs credentials to access private repositories.
- Store credentials securely in Jenkins.
- Types:

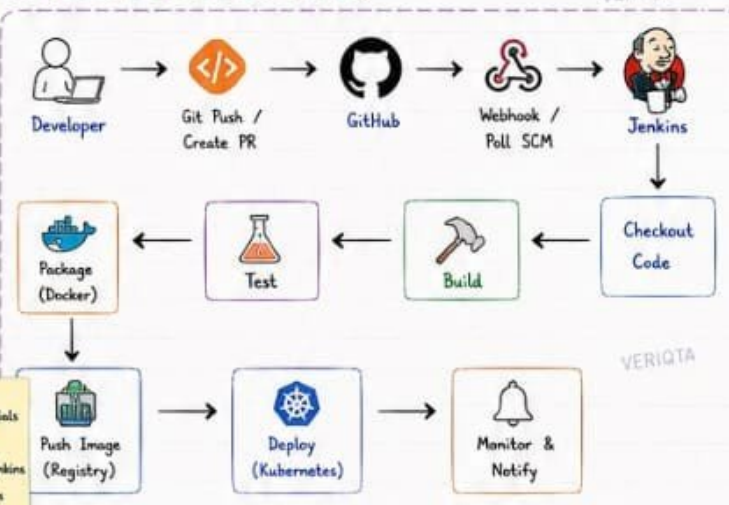
- ☒ Username with password
- ☒ SSH Username with private key
- ☒ Personal Access Token (PAT)
- ☒ GitHub App (advanced)

BEST PRACTICES

- ☒ Use least privileged access
- ☒ Use SSH keys over passwords
- ☒ Rotate tokens regularly
- ☒ Never hardcode credentials
- ☒ Limit repo access

Store credentials at: Jenkins → Manage Jenkins → Credentials

7. PIPELINE FLOW (END TO END)



QUICK CONFIG NOTES

Webhook URL (GitHub)

`http[s]://JENKINS_URL/github-webhook/`
(or)

`http[s]://JENKINS_URL/generic-webhook-trigger/invoke?token=YOUR_TOKEN`

Enable Webhook in GitHub

1. Repository → Settings
2. Webhooks → Add webhook
3. Payload URL → Paste Jenkins URL
4. Content type → application/json
5. Select events → Let Jenkins decide

Multibranch Pipeline

- ☒ Auto-detects branches
- ☒ Auto-discovers PRs
- ☒ Scales for teams
- ☒ Best for modern setups



KEY TAKEAWAY

Good Git integration = Faster feedback + Better quality + Reliable deployments!





8. BUILDING DOCKER IMAGES

Jenkins builds Docker images, tags them, pushes to a registry (DockerHub or ECR) and promotes them through environments.



Goal: Build once,
Run anywhere!

1. DOCKER BUILD

- Build image from Dockerfile.
- Creates immutable image layer by layer.

```
# Build image
docker build -t myapp:1.0.0
```



DOCKERFILE

```
# Build with no cache
docker build --no-cache -t myapp:1.0.0
```

2. DOCKER TAG

- Tag image for versioning and registry.

```
# Tag for DockerHub
docker tag myapp:1.0.0 \
youruser/myapp:1.0.0
```

```
# Tag for ECR
docker tag myapp:1.0.0 \
123456789012.dkr.ecr.us-east-1.
amazonaws.com/myapp:1.0.0
```

3. DOCKER PUSH

- Push image to remote registry.

```
# Push to DockerHub
docker push youruser/myapp:1.0.0

# Push to ECR
docker push 123456789012.dkr.ecr.
us-east-1.amazonaws.com/myapp:1.0.0
```

Tip

Always use versioned tags
(1.0.0, 1.0.1) instead of
latest in production.

4. DOCKERHUB

- Public or private repositories.
- Easy to use and integrate.
- Great for small/medium projects.

5. AMAZON ECR

- Private, secure, highly available.
- Integrated with IAM.
- Best for AWS workloads.

6. MULTI-STAGE BUILDS

- Smaller images, faster builds, better security.
- Only final artifacts in runtime image.

Useful Commands

```
docker login
docker logout
docker search nginx
docker pull nginx:alpine
```

Useful Commands

```
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password stdin \
123456789012.dkr.ecr.us-east-1.amazonaws.com

aws ecr create-repository --repository-name myapp
```

Benefits

- ☒ Smaller image size
- ☒ No build tools in final image
- ☒ Better security
- ☒ Faster deployments

Example: Multi-stage Dockerfile

```
# ----- Build Stage -----
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# ----- Runtime Stage -----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/myapp.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

7. SECURITY SCANNING

- Scan images for vulnerabilities before pushing.
- Tools: Trivy, Gype, Snyk, Docker Scout.

Example with Trivy

```
# Scan local image
trivy image myapp:1.0.0

# Fail build on HIGH and CRITICAL
trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:1.0.0
```

What it checks

- ★ OS packages
- ★ Application dependencies
- ★ Secrets
- ★ Misconfigurations

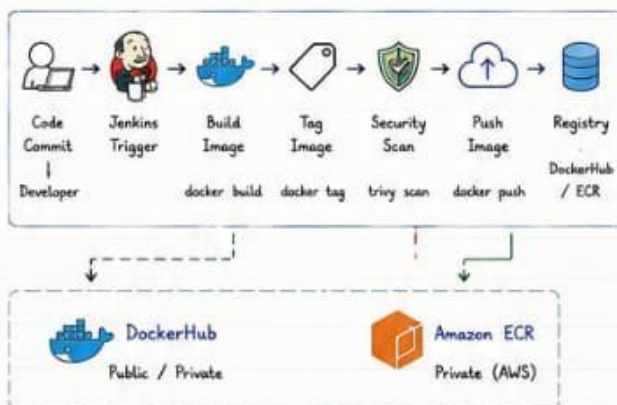
8. BEST PRACTICES

- ☒ Use specific base images (alpine, slim)
- ☒ Use .dockerignore
- ☒ Tag with version (semver)
- ☒ Scan image before pushing
- ☒ Don't run as root user
- ☒ Keep images small
- ☒ Use multi-stage builds
- ☒ Remove unused packages
- ☒ Store credentials securely
- ☒ Automate in Jenkins pipeline

.dockerignore (Example)

```
.git
target/
node_modules/
*.log
Dockerfile
docker-compose.yml
```

9. DOCKER PIPELINE FLOW



10. JENKINS PIPELINE EXAMPLE (DOCKER STAGE)

```
01 stage('Docker Build & Push') {
02   steps {
03     script {
04       def image = "myapp:${env.BUILD_NUMBER}"
05       sh "docker build -t $image ."
06       sh "docker tag $image ${env.DOCKERHUB_USER}/${myapp:$image}"
07       withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', {
08         usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS'})]) {
09         sh "echo $DOCKER_PASS | docker login -u $DOCKER_USER --password stdin"
10         sh "docker push ${env.DOCKERHUB_USER}/${myapp:$image}"
11         sh "docker logout"
12       }
13     }
14   }
15 }
16 }
```

Explanation

1. Build image
2. Tag image
3. Login to DockerHub using stored credentials
4. Push image
5. Logout (good practice)

Pro Tips

- ☒ Pin base image versions.
- ☒ Scan before push.
- ☒ Use immutable tags.
- ☒ Automate cleanup of old images.
- ☒ Use ECR for production workloads.

BUILD ONCE → SCAN → PUSH → DEPLOY ANYWHERE
Small Images. Secure Images. Reliable Deployments.



9. JENKINS + KUBERNETES

Jenkins can run on Kubernetes and also use Kubernetes as a dynamic, scalable build infrastructure.

1. KUBERNETES AGENTS



- Jenkins agents run as Pods in Kubernetes.
- Each build can use a fresh pod with all tools.
- No need to manage static agents.
- Scales automatically based on workload.

TYPES OF AGENTS

- Static Agent (VM / Bare metal)
- Cloud Agent (EC2, VM, etc)
- Kubernetes Agent (Dynamic Pod)

2. DYNAMIC PODS



- Jenkins creates a pod on demand when a build starts.
- Pod runs the build, then terminates after completion.
- Defined using YAML/ Pod Template.

DYNAMIC POD TEMPLATE (EXAMPLE)

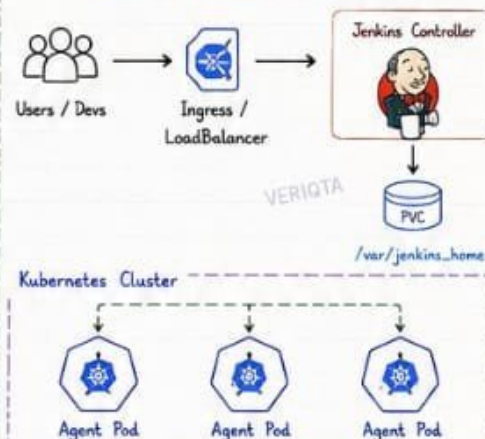
```
apiVersion: v1
kind: Pod
metadata:
  labels:
    jenkins: dynamic-agent
spec:
  containers:
    - name: jnlp
      image: jenkins/inbound-agent:latest
      args: ["$(JENKINS_SECRET)", "$(JENKINS_NAME)"]
      resources:
        limits:
          cpu: "500m"
          memory: "512Mi"
  volumes:
    - name: workspace
      emptyDir: {}
```

3. JENKINS ON KUBERNETES



- Jenkins Controller can also run in a Pod.
- Use StatefulSet for persistence.
- Persistent Volume for /var/jenkins_home.

JENKINS ON K8s (HIGH LEVEL)



4. BUILD PODS



- Each build runs in a dedicated Pod.
- Tools (maven, node, a docker, kubectl, etc) are defined in the pod.
- Clean, isolated, reproducible builds.

EXAMPLE BUILD POD (MAVEN)

```
spec:
  containers:
    - name: maven
      image: maven:3.9-eclipse-temurin-17
      command: ["cat"]
      tty: true
      volumeMounts:
        - name: m2-repo
          mountPath: /root/.m2
  volumes:
    - name: m2-repo
      emptyDir: {}
```

Tools inside the container

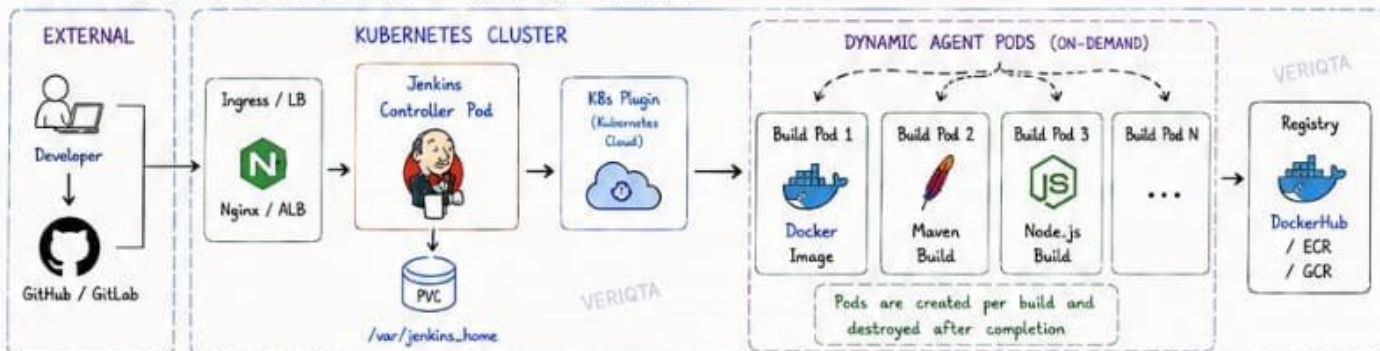
5. AGENT LIFECYCLE



Why Ephemeral Agents?

- ✓ No manual cleanup
- ✓ Consistent environments
- ✓ Better security isolation
- ✓ Scale to hundreds of builds
- ✓ Pay only for what you use

6. ARCHITECTURE DIAGRAM (JENKINS + KUBERNETES AGENTS)



7. ESSENTIAL REQUIREMENTS

- ✓ Kubernetes cluster (v1.20+)
- ✓ Jenkins with Kubernetes Plugin
- ✓ RBAC permissions for Jenkins
- ✓ ServiceAccount for Jenkins
- ✓ Persistent storage for controller
- ✓ Container registry credentials



8. BEST PRACTICES

- ★ Use resource requests/limits for pods.
- ★ Use minimal base images for faster startup.
- ★ Mount caches (Maven repo, npm cache) using volumes.
- ★ Use labels & selectors for routing builds.
- ★ Keep controller and agents in same cluster.
- ★ Monitor pod usage & set quotas.



9. USE CASES

- ✓ Microservices CI/CD
- ✓ Monorepo builds
- ✓ Large parallel builds
- ✓ Multi-language projects
- ✓ On-demand ephemeral environments





10. CI/CD PIPELINE

COMPLETE PRODUCTION PIPELINE



Automate everything. Deliver value faster.

Build → Test → Secure → Deploy → Monitor

END-TO-END FLOW

- 1 **DEVELOPER**
Writes code and commits changes.
 - 2 **GITHUB**
Code is pushed to GitHub repository.
 - 3 **WEBHOOK**
GitHub sends an HTTP POST request to Jenkins.
 - 4 **JENKINS**
Jenkins receives the event and starts the pipeline.
 - 5 **BUILD**
Compile source code and package the app.
 - 6 **UNIT TESTS**
Run unit tests to verify code functionality.
 - 7 **SONARQUBE**
Analyze code quality, bugs and coverage.
 - 8 **DOCKER BUILD**
Build Docker image from Dockerfile.
 - 9 **TRIVY SCAN**
Scan image for vulnerabilities and misconfigurations.
 - 10 **DOCKER REGISTRY**
Push the secure image to Docker Registry.
 - 11 **DEPLOY**
Deploy application using Kubernetes manifests.
 - 12 **KUBERNETES**
Kubernetes pulls image and runs the containers.
 - 13 **MONITORING**
Monitor the application, logs, metrics and alerts.
- ✓ **PIPELINE SUCCESS**
Code → Production Automatically!

SAMPLE JENKINSFILE (OVERVIEW)

```

1 pipeline {
2   agent any
3   environment {
4     DOCKER_IMAGE = "myapp:${env.BUILD_NUMBER}"
5   }
6   stages {
7     stage('Build') { steps { sh 'mvn clean package' } }
8     stage('Unit Tests') { steps { sh 'mvn test' } }
9     stage('SonarQube') { steps {
10       withSonarQubeEnv('SonarQube') { sh 'mvn sonar:sonar' }
11     } }
12     stage('Docker Build') { steps { sh 'docker build -t $DOCKER_IMAGE .' } }
13     stage('Trivy Scan') { steps { sh 'trivy image $DOCKER_IMAGE --exit-code 1' } }
14     stage('Push Image') { steps { sh 'docker push $DOCKER_IMAGE' } }
15     stage('Deploy') { steps { sh 'kubectl apply -f k8s/' } }
16   }
17   post {
18     success { echo 'Pipeline Succeeded' }
19     failure { echo 'Pipeline Failed' }
20   }
21 }

```

This is a simplified example

WHAT IS CI/CD?

CI (Continuous Integration):

Developers integrate code frequently.

CD (Continuous Delivery):

Code is always ready to be released.

Continuous Deployment:

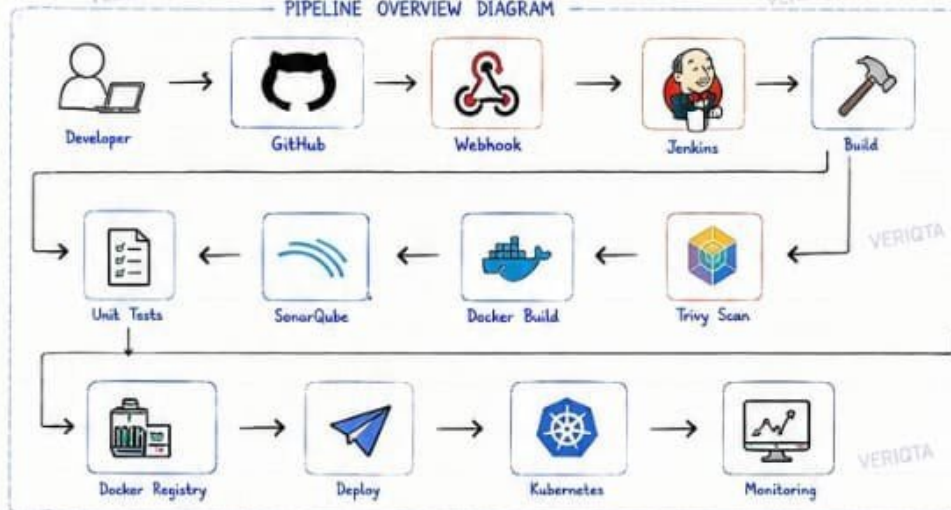
Code is automatically released to production.



KEY BENEFITS

- ✓ Faster delivery
- ✓ Early bug detection
- ✓ High code quality
- ✓ Consistent releases
- ✓ Reduced manual errors
- ✓ Better collaboration
- ✓ Full automation

PIPELINE OVERVIEW DIAGRAM



PIPELINE STAGES EXPLAINED

1. Developer: Write and commit code.
2. GitHub: Stores code in a version control system.
3. Webhook: Notifies Jenkins of code changes.
4. Jenkins: Orchestrates the complete pipeline.
5. Build: Compiles code and creates artifacts.
6. Unit Tests: Validates the functionality.
7. SonarQube: Ensures code quality and security.
8. Docker Build: Packages app into Docker image.
9. Trivy Scan: Scans image for vulnerabilities.
10. Docker Registry: Stores and manages images.
11. Deploy: Deploys the application to environment.
12. Kubernetes: Runs and manages containers.
13. Monitoring: Monitors logs, metrics and health.

QUALITY GATES

- ✓ Build Success
- ✓ All Tests Pass
- ✓ Code Quality OK
- ✓ No Critical Vulnerabilities

If any gate fails → Pipeline stops



TOOLS USED

- GitHub
- Jenkins
- Maven / Gradle
- JUnit / TestNG
- SonarQube
- Docker
- Trivy
- DockerHub / ECR
- Kubernetes
- Prometheus + Grafana

FAILED PIPELINE EXAMPLE



SUCCESS PIPELINE EXAMPLE



BEST PRACTICES

- ★ Use version control for everything.
- ★ Keep builds repeatable and idempotent.
- ★ Run tests early and often.
- ★ Scan for vulnerabilities before deploying.
- ★ Use immutable Docker images.
- ★ Promote the same artifact through environments.
- ★ Use infrastructure as code (IaC).
- ★ Monitor everything in production.
- ★ Keep credentials secure.
- ★ Use quality gates to enforce standards.



PRO TIP

Start small, automate one thing at a time.
Keep your pipeline fast, reliable and observable.
Your pipeline is your competitive advantage!

COMMON PITFALLS

- ⚠ Skipping tests
- ⚠ Hardcoded credentials
- ⚠ Manual deployments
- ⚠ No rollback strategy
- ⚠ Ignoring security scans
- ⚠ Overly complex pipelines
- ⚠ Not monitoring in production





11. SECRETS MANAGEMENT



Secrets are sensitive information used by Jenkins and pipelines.
Never hardcode secrets in code or Jenkinsfiles!

WHY IT MATTERS

- ✓ Protects sensitive data from exposure
- ✓ Prevents unauthorized access
- ✓ Keeps pipelines secure and compliant
- ✓ Enables safe automation



1. JENKINS CREDENTIALS OVERVIEW

Jenkins Credentials store secrets securely and provide them to jobs/pipelines when required. They are encrypted at rest.



WHERE ARE CREDENTIALS STORED?

- Jenkins Credentials Store
- Stored encrypted on disk
(\$JENKINS_HOME/credentials.xml)
- Access controlled by Jenkins permissions

2. TYPES OF CREDENTIALS

SECRET TEXT

For tokens, API keys, license keys, etc.

Example Use Cases:

- API Tokens
- Slack Webhook URL
- License Keys

Example (Jenkinsfile)

```
withCredentials([string(
  credentialsId: 'my-token',
  variable: 'TOKEN'
)]) {
  sh 'echo $TOKEN'
}
```

USERNAME + PASSWORD

For services requiring username and password.

Example Use Cases:

- Docker Registry
- Database Login
- SonarQube, Nexus

Example (Jenkinsfile)

```
withCredentials([usernamePassword(
  credentialsId: 'docker-creds',
  usernameVariable: 'DOCKER_USER',
  passwordVariable: 'DOCKER_PASS'
)]) {
  sh 'docker login -u $DOCKER_USER -p $DOCKER_PASS'
}
```

SSH KEYS

For authenticating to Git, servers or remote systems.

Example Use Cases:

- Git over SSH
- Deploy to Servers
- Ansible / Terraform

Example (Jenkinsfile)

```
withCredentials([sshUserPrivateKey(
  credentialsId: 'deploy-key',
  keyFileVariable: 'SSH_KEY',
  usernameVariable: 'SSH_USER'
)]) {
  sh 'ssh -i $SSH_KEY $SSH_USER@server "uptime"'
}
```

AWS CREDENTIALS

For accessing AWS services securely.

Example Use Cases:

- AWS CLI
- Terraform
- Deploy to ECR, S3, EKS

Example (Jenkinsfile)

```
withCredentials([class:
  'AmazonWebServicesCredentialsBinding',
  credentialsId: 'aws-creds']) {
  sh 'aws s3 ls'
}
```

BINDINGS / FILE

For certificates, config files, or custom files.

Example Use Cases:

- Kubeconfig
- TLS Certificates
- Service Account JSON

Example (Jenkinsfile)

```
withCredentials([file(
  credentialsId: 'kubeconfig',
  variable: 'KUBECONFIG'
)]) {
  sh 'kubectl get pods --kubeconfig $KUBECONFIG'
}
```

3. USING CREDENTIALS IN JENKINSFILE

```
1 pipeline {
2   agent any
3   environment {
4     DOCKER_REGISTRY = 'registry.veriqta.com'
5   }
6   stages {
7     stage('Login to Docker') {
8       steps {
9         withCredentials([usernamePassword(
10           credentialsId: 'docker-creds',
11           usernameVariable: 'DOCKER_USER',
12           passwordVariable: 'DOCKER_PASS'
13         )]) {
14           sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER \
15             --password-stdin $DOCKER_REGISTRY'
16         }
17       }
18     }
19     stage('Use Secret Text') {
20       steps {
21         withCredentials([string(credentialsId: 'my-token', variable: 'TOKEN')]) {
22           sh 'curl -H "Authorization: Bearer $TOKEN" https://api.veriqta.com/data'
23         }
24       }
25     }
26   }
27 }
```

DO NOT DO THIS

Hardcoding secrets is dangerous!

```
sh 'docker login -u admin -p mysecret123'
sh 'aws configure set aws_access_key_id AKIA...'
sh 'export TOKEN=abcd1234'
```

GOOD PRACTICE

- ★ Always use credentials.
- ★ Use least privilege access.
- ★ Rotate secrets regularly.
- ★ Store secrets in Vault or cloud secret managers when possible.

4. VAULT INTEGRATION



Use Vault to store and retrieve secrets dynamically.

Benefits:

- Centralized secret store
- Dynamic secrets (temporary)
- Audit logs
- Fine-grained access control

Example (with HashiCorp Vault Plugin)

```
withVault([vaultSecrets: [
  [path: 'secret/data/ci',
   secretValues: [
     [envVar: 'DB_PASSWORD',
      vaultKey: 'password']]
  ]
]]) {
  sh 'echo $DB_PASSWORD'
}
```

5. BEST PRACTICES

- ✓ Never store secrets in code, logs, or environment variables.
- ✓ Use Jenkins Credentials for all secrets.
- ✓ Limit credential scope to the minimum necessary.
- ✓ Use folders & permissions to restrict access.
- ✓ Rotate secrets regularly.
- ✓ Mask secrets in logs by default.
- ✓ Use external secret managers (Vault, AWS Secrets Manager) for critical environments.
- ✓ Review and audit credentials periodically.
- ✓ Remove unused credentials.



6. COMMON MISTAKES

- ✗ Hardcoding passwords/tokens in Jenkinsfile
- ✗ Using admin/root credentials
- ✗ Overprovisioned AWS keys
- ✗ Sharing credentials across teams
- ✗ Printing secrets in logs (echo \$PASSWORD)
- ✗ Not revoking old credentials
- ✗ Not using credentials for Docker registry / Git / AWS



SECURE SECRETS = SECURE PIPELINES = SECURE PRODUCTION

100



12. JENKINS SECURITY

Security is not optional. Secure Jenkins = Secure Pipelines = Secure Production.

SECURITY GOALS

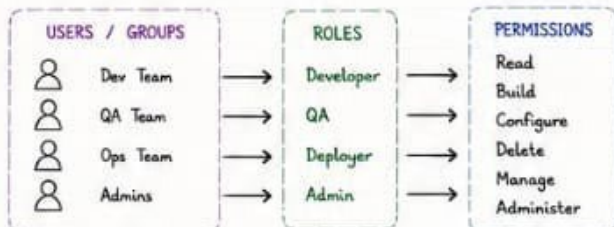
- ✓ Protect Jenkins from unauthorized access
- ✓ Protect credentials and sensitive data
- ✓ Secure agents and build environments
- ✓ Prevent attacks and misconfigurations
- ✓ Ensure auditability and compliance



1. RBAC (ROLE BASED ACCESS CONTROL)



- Grant permissions based on roles.
- Avoid "admin" access for everyone.



Tip: Use groups from your IdP (LDAP / AD) for better control.

2. MATRIX AUTHORIZATION



- Fine-grained permissions for every user or group.
- Control who can do what.

User / Group	Read	Build	Configure	Delete	Administer
Dev Team	✓	✓	✗	✗	✗
QA Team	✓	✓	✗	✗	✗
Ops Team	✓	✓	✓	✓	✗
Admins	✓	✓	✓	✓	✓

Path: Manage Jenkins → Configure Global Security
→ Authorization → Matrix-based Security

3. CSRF PROTECTION



- Prevents Cross Site Request Forgery attacks.
- Should always be enabled.

What is CSRF?

An attacker tricks an authenticated user's browser to perform unwanted actions on Jenkins.



Path: Manage Jenkins → Configure Global Security
→ Enable CSRF Protection

4. AGENT SECURITY



- Agents run your build jobs. Secure them!
- Use dedicated agents for untrusted jobs.
- Keep agents updated and minimal.
- Do not allow arbitrary code on agents.

Best Practices

- ✓ Use inbound agents (no open SSH)
- ✓ Limit tools and permissions
- ✓ Run agents as non-root users
- ✓ Isolate agent networks (VPC/Subnet)
- ✓ Auto-remove ephemeral agents



5. CREDENTIAL PROTECTION



- Never store secrets in code or logs.
- Use Jenkins Credentials Store.
- Mask secrets in console output.
- Restrict who can use credentials.

Credentials Storage



- ✓ Encrypted at rest
- ✓ Encrypted in transit
- ✓ Access controlled
- ✓ Scoped to folders / jobs

Never use: hardcoded passwords, tokens, API keys in Jenkinsfile or scripts!

6. LEAST PRIVILEGE PRINCIPLE



- Give users and processes the minimum access needed to perform their tasks.
- Reduce blast radius.

DO

- ✓ Use specific roles
- ✓ Limit admin access
- ✓ Review permissions regularly
- ✓ Use folders for isolation

DON'T

- ✗ Avoid "admin" for everyone
- ✗ Don't share accounts
- ✗ Don't over-permission service accounts



7. PRODUCTION HARDENING



- ✓ Use HTTPS with valid TLS certificate
- ✓ Disable HTTP
- ✓ Enable login with IdP (LDAP/AD/OAuth)
- ✓ Disable sign up
- ✓ Keep Jenkins and plugins updated
- ✓ Backup Jenkins (jobs, configs, plugins)
- ✓ Disable unused plugins
- ✓ Limit build disc space usage
- ✓ Audit logs regularly
- ✓ Monitor Jenkins health
- ✓ Use reverse proxy (Nginx/Apache)
- ✓ Set resource limits (CPU/Memory)



8. SECURITY CHECKLIST



- ✓ RBAC configured
- ✓ Matrix Authorization reviewed
- ✓ CSRF Protection enabled
- ✓ Credentials stored securely
- ✓ Agents secured and isolated
- ✓ Least privilege enforced
- ✓ Jenkins & plugins up to date
- ✓ Backups configured
- ✓ Audit logs enabled
- ✓ Monitoring & alerting enabled
- ✓ Security headers enabled
- ✓ Regular vulnerability scanning



ATTACK SURFACE TO PROTECT



Internet Exposure



Weak Credentials



Vulnerable Plugins



Leaked Secrets



Compromised Agents



Misconfigs



Remember:

A secure Jenkins today prevents a production incident tomorrow. Security is everyone's responsibility.

TOP SECURITY TIPS

- ★ Lock it down
- ★ Keep it updated
- ★ Monitor it
- ★ Back it up
- ★ Audit it
- ★ Test it
- ★ Document it
- ★ Improve it



USEFUL SECURITY COMMANDS

VERIQTA

```
# Generate an API token (User Settings → API Tokens)
# Or via groovy script
println(Jenkins.instance.getId())

# Backup Jenkins
tar -czf jenkins-backup-$(date +%F).tar.gz $JENKINS_HOME

# Check Jenkins version
curl -s http://localhost:8080/api/json | jq .version

# List installed plugins
curl -s http://localhost:8080/pluginManager/api/json?depth=1 | jq .plugins[].shortName
```

SECURE JENKINS = SECURE PIPELINES = SECURE PRODUCTION

100



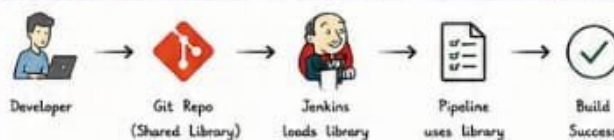


13. SHARED LIBRARIES



Shared Libraries allow you to share common code, steps, functions and resources across multiple Pipelines.

HOW IT WORKS



WHY USE SHARED LIBRARIES?

- ✓ DRY Principle - Don't Repeat Yourself
- ✓ Consistency across all Pipelines
- ✓ Centralized updates (update once, use everywhere)
- ✓ Easier maintenance & version control
- ✓ Enforce best practices & standards
- ✓ Reduce duplication and boilerplate code
- ✓ Enable enterprise-grade DevOps
- ✓ Speed up development



TYPES OF SHARED LIBRARIES

- Global Library - Available to all Pipelines on the Jenkins instance.
- Folder-specific Library - Available only to Pipelines in that folder.

USAGE IN JENKINSFILE

```

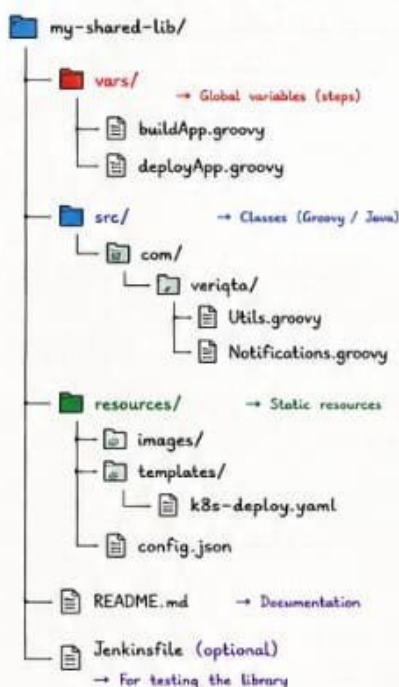
@Library('my-shared-lib@main') _
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                myLib.buildApp()
            }
        }
    }
}
  
```

BENEFITS IN ENTERPRISE

- ★ Standardized CI/CD
- ★ Faster onboarding
- ★ Governance & compliance
- ★ Audit & traceability
- ★ Scalable for large teams



FOLDER STRUCTURE OF A SHARED LIBRARY



vars/

- Contains Groovy scripts that define global variables.
- Each .groovy file becomes a step in your Pipeline.
- The filename = step name.

Example:
vars/buildApp.groovy
→ step: buildApp()



src/

- Contains classes written in Groovy or Java.
- Used for complex logic, objects, utilities.
- Must be in a package (folder structure).

Example:
src/com/veriqta/Utils.groovy
→ new com.veriqta.Utils()



resources/

- Stores non-code resources.
- Access in Pipeline using libraryResource() or readYaml(), readJSON().
- Useful for YAML templates, config files, images, etc.



EXAMPLE STEP (vars/buildApp.groovy)

```

def call(Map config = [:]) {
    echo "Building ${config.appName}..."
    sh "mvn clean package -DskipTests"
}
  
```



EXAMPLE CLASS (src/com/veriqta/Utils.groovy)

```

package com.veriqta
class Utils implements Serializable {
    def static String getVersion() {
        return '1.0.0'
    }
}
  
```



EXAMPLE RESOURCE (resources/templates/k8s-deploy.yaml)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: myapp
          image: myapp:latest
  
```



HOW TO CONFIGURE IN JENKINS

1. Go to: Manage Jenkins → Configure System
2. Scroll to: Global Pipeline Libraries
3. Add Library
 - Name: my-shared-lib
 - Default Version: main
 - Source Code Management: Git
 - Repository URL: <https://github.com/your-org/my-shared-lib.git>
 - Credentials: (select your git credentials)
4. Save

JENKINSFILE EXAMPLE

```

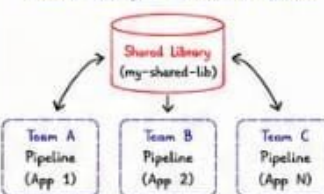
@Library('my-shared-lib@main') _
import com.veriqta.Utils

pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                buildApp(appName: 'myapp')
                echo "Version: ${Utils.getVersion()}"
            }
        }
        stage('Deploy') {
            steps {
                deployApp(env: 'prod')
            }
        }
    }
}
  
```



ENTERPRISE PIPELINES

Central Library → Standard Pipelines



- ✓ One Library, many teams
- ✓ One update, everyone benefits
- ✓ Enforce standards & security
- ✓ Scale DevOps across org

REUSABLE CODE EXAMPLES

- Common Steps**
buildApp(), deployApp(), runTests(), sendSlack()
- Utilities**
Utils.groovy, StringUtils.groovy, K8sHelper.groovy
- Templates**
K8s YAML, Dockerfile, Helm values, JSON configs
- Resources**
Logos, scripts, policy files, text templates

BEST PRACTICES

- ✓ Keep libraries small and modular.
- ✓ Version your libraries (use branches or tags).
- ✓ Write documentation (README.md).
- ✓ Test your library with a Jenkinsfile.
- ✓ Follow semantic versioning (v1.0.0, v1.1.0).
- ✓ Review changes before merging.
- ✓ Restrict who can modify the library.



THINGS TO AVOID

- ✗ Huge libraries doing too many things
- ✗ Breaking changes without versioning
- ✗ Hardcoding environment values
- ✗ Overuse of global variables
- ✗ Storing secrets in library code



QUICK REFERENCE

Component	Purpose
vars/	Create custom steps
src/	Groovy/Java classes
resources/	Static files, templates
@Library	Load library in Jenkinsfile
libraryResource()	Read resource file
new Class()	Use class from src/

Remember:

Shared Library
= Reusable Code
+ Standardization
+ Scalability
= Enterprise DevOps





14. PRODUCTION TROUBLESHOOTING



Problems will happen. Smart engineers fix fast and prevent next time.

Observe → Diagnose → Resolve → Prevent

TROUBLESHOOTING MINDSET

- ✓ Check the facts, not assumptions
- ✓ Look at logs first
- ✓ Isolate the problem
- ✓ Fix the root cause, not the symptom
- ✓ Document & share the solution



1. COMMON PRODUCTION ISSUES



1. BUILD HANGS Build runs forever without finishing. Impact: High	2. AGENT OFFLINE Agents go offline or disconnected. Impact: High	3. WORKSPACE CORRUPTION Files missing, permissions broken, or corrupted. Impact: Medium	4. PLUGIN CONFLICTS Incompatible plugins cause failures. Impact: Medium
5. BUILD QUEUE STUCK Jobs stuck in queue and not starting. Impact: High	6. MEMORY ISSUES Out of memory, slow performance. Impact: High	7. JAVA CRASHES Jenkins/Agent JVM crashes or exits. Impact: High	8. DISK SPACE FULL No space left for logs, workspace or builds. Impact: High

2. REAL PRODUCTION CASES



CASE	SYMPTOM	ROOT CAUSE	SOLUTION
Build Hangs	Stage never completes	Shell script waiting for input	Add timeout & non-interactive commands
Agent Offline	Agent shows offline	Network issue / Agent crashed	Restart agent, check network, logs
Workspace Corruption	File not found, permission denied	Interrupted build / Bad cleanup	Clean workspace, fix permissions
Plugin Conflicts	Jenkins fails to start	Incompatible plugins	Update/Remove conflicting plugins
Queue Stuck	Jobs pending forever	No executors / Deadlock	Increase executors, restart queue
Memory Issues	Jenkins slow or kills jobs	Low heap size / Memory leak	Increase heap, analyze memory
Java Crashes	Jenkins/Agent crashes	JVM error / OOM / Bug	Check logs, upgrade Java

3. QUICK CHECKLIST



- ✓ Check Jenkins system log
- ✓ Check build console output
- ✓ Check agent logs (agent.log)
- ✓ Check disk space & memory
- ✓ Check network connectivity
- ✓ Check plugin health
- ✓ Check recent changes/configs
- ✓ Reproduce in a test job
- ✓ Document findings

IMPORTANT LOG LOCATIONS

Jenkins Controller:
\$JENKINS_HOME/logs/jenkins.log
Agent:
\$JENKINS_HOME/logs/agent.log
Build Console: Job → Console Output
System Info: Manage Jenkins → System Information



PRO TIP Always reproduce the issue in a safe environment before applying fixes in production.



5. PERFORMANCE & RESOURCE GUIDELINES



RESOURCE	RECOMMENDATION (MIN)
CPU (Controller)	2 vCPU
RAM (Controller)	4 - 8 GB (min) 8 - 16 GB (recommended)
Disk Space	50 GB+ (SSD Recommended)
Heap Size (Jenkins JVM)	-Xms2g -Xmx4g (example)
Agent Executors	# of executors = # of CPU cores

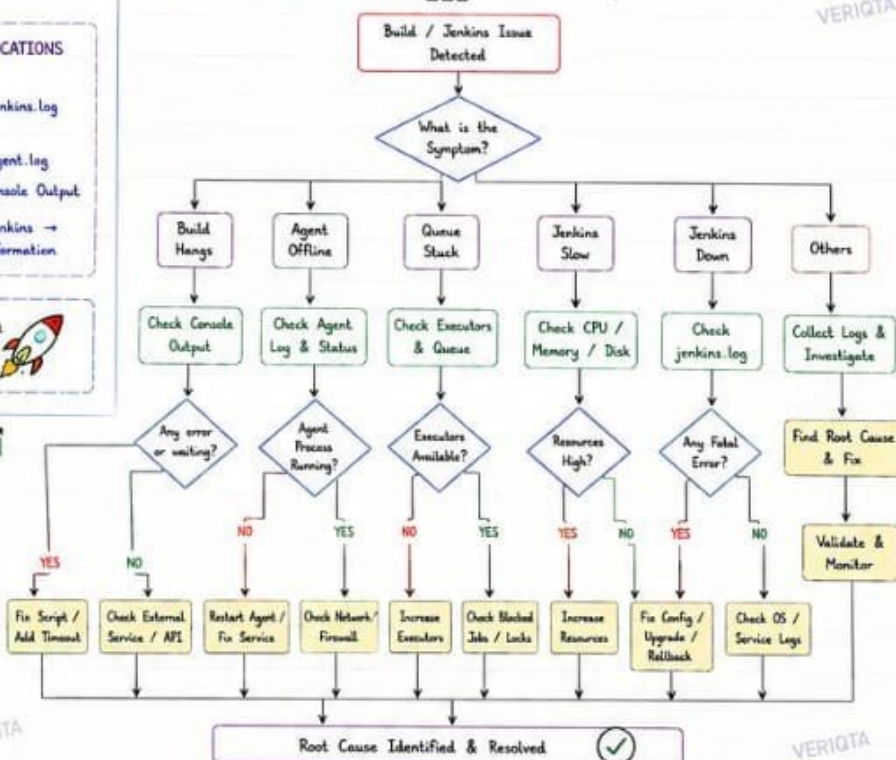


Heap Size Rule of Thumb

-Xmx = 50% of available RAM

(Do not exceed 8GB without strong reason)

4. DIAGNOSIS FLOWCHART



6. PREVENTION BEST PRACTICES



- ☆ Monitor Jenkins (CPU, RAM, Disk, Builds, Queue)
- ☆ Use log rotation to prevent disk full
- ☆ Keep Jenkins & plugins up to date
- ☆ Use stable versions in production
- ☆ Regular backups (config, jobs, plugins)
- ☆ Test changes in staging before production
- ☆ Use health checks & alerts
- ☆ Document solutions for known issues



7. USEFUL COMMANDS



- ☆ Check Jenkins status
systemctl status jenkins
- ☆ Restart Jenkins
systemctl restart jenkins
- ☆ Check agent status
java -jar agent.jar -jnlpUrl <url> -secret <secret>
- ☆ Check disk space
df -h
- ☆ Check memory usage
free -h

8. MONITORING & ALERTING



METRIC	TOOL	ALERT WHEN
CPU Usage	Prometheus / Grafana	> 80%
Memory Usage	Prometheus / Grafana	> 80%
Disk Usage	Prometheus / Grafana	> 80%
Build Failures	Prometheus / Alertmanager	> 5 in 15m
Queue Length	Prometheus / Grafana	> 10
Agent Offline	Jenkins Health Plugin	Any Agent Offline



OBSERVE CAREFULLY. DIAGNOSE QUICKLY. FIX CORRECTLY. PREVENT ALWAYS. **100**





15. PERFORMANCE OPTIMIZATION



Faster builds = Happier developers + Lower costs + Better delivery speed.
Optimize continuously. Measure always.

VERIQTA

PERFORMANCE PRINCIPLES

- ✓ Eliminate waste
- ✓ Run in parallel
- ✓ Cache everything you can
- ✓ Clean what you don't need
- ✓ Distribute the load



FASTER = BETTER

1. EXECUTORS (PARALLELISM)



- Executors define how many builds run simultaneously on a node.
- More executors = more throughput (but more resource usage).

WHERE TO CONFIGURE

- Manage Jenkins → Configure System
- # of executors (global)
- Or on Agent (Node) level

EXECUTOR BEST PRACTICES

- ☆ Don't set too high (CPU/RAM exhaustion).
- ☆ Start with 1 executor per CPU core.
- ☆ Use labels to distribute workload.
- ☆ Separate heavy jobs from light jobs.

EXAMPLE

Controller (Light jobs)
Executors: 2

Agent (Heavy jobs)
Executors: 8
Label: heavy



2. PARALLEL BUILDS



- Run multiple stages or jobs in parallel.
- Use Declarative Parallel or Matrix builds.

DECLARATIVE PARALLEL EXAMPLE

```
stage('Build & Test') {
  parallel {
    stage('Unit Tests') { steps { sh 'mvn test' } }
    stage('Integration Tests') { steps { sh 'mvn verify' } }
    stage('Lint') { steps { sh 'npm run lint' } }
  }
}
```

BENEFITS

- ☆ Reduces total build time
- ☆ Better resource utilization
- ☆ Faster feedback to developers



3. BUILD CACHE



Cache dependencies, tools and build outputs.
Speeds up repeat builds dramatically.

WHAT TO CACHE	TOOLS / EXAMPLES	HOW
Dependencies	Maven, Gradle, NPM, Pip	Cache plugin / tool specific cache
Build Outputs	.m2, .gradle, node_modules	Cache plugin
Docker Layers	Docker build layers	Docker layer cache
Artifacts	JARs, WARs, NPM packages	Archive + reuse
Tool Installations	JDK, Maven, Node	Global tool cache

JENKINS CACHE PLUGIN

- ✓ Easy to configure
- ✓ Key based caching
- ✓ Supports includes / excludes
- ✓ Works with Pipeline

Example:

```
cache {
  includes: '**/.m2/**',
  key: "maven-${env.JOB_NAME}"
}
```



4. DOCKER CACHE



- Reuse Docker layers.
- Avoid rebuilding unchanged layers.

BEST PRACTICES

- ☆ Order Dockerfile from least changing to most changing.
- ☆ Use .dockerignore
- ☆ Leverage BuildKit
- ☆ Push Cache to Registry (BuildKit)

Example (BuildKit Cache)

```
docker buildx build \
  --cache-from=type=registry,ref=myapp:cache \
  --cache-to=type=registry,ref=myapp:cache,mode=max \
  -t myapp:latest
```

5. WORKSPACE CLEANUP



- Clean old workspaces and unused files.
- Prevent disk full and slow builds.

WAYS TO CLEAN

- ☆ Clean Before Checkout (Plugin)
- ☆ Clean Workspace (Post Build Action)
- ☆ Delete Dir() in Pipeline
- ☆ Periodic Workspace Wipe
- ☆ Use 'ws-cleanup' plugin



PIPELINE EXAMPLE

```
post {
  always {
    cleanWs() // clean workspace
  }
}
```



6. BUILD RETENTION



- Keep only what you need.
- Old builds waste disk and slow UI.

CONFIGURE (MANAGE JENKINS → SYSTEM CONFIGURATION)

Discard Old Builds

- ✓ Days to keep 30
- ✓ Max # of builds 50
- ✓ Artifact days to keep 30

Tip: Keep logs short & artifacts only as long as required.

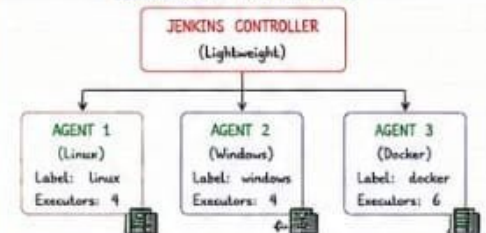


7. DISTRIBUTED AGENTS



- Distribute builds across multiple agents/nodes.
- Use labels to route jobs to right nodes.

DISTRIBUTED ARCHITECTURE



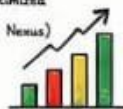
BENEFITS

- ✓ Scales horizontally
- ✓ Isolates heavy jobs
- ✓ Faster builds
- ✓ Better resource utilization

8. ADDITIONAL OPTIMIZATION TIPS

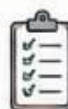


- ✓ Use lightweight checkout (Shallow Clone)
- ✓ Avoid unnecessary steps (Echo, Sleep, etc.)
- ✓ Run tests in parallel (Surefire, Jest, Pytest-xdist)
- ✓ Use ephemeral agents for clean environments
- ✓ Turn off concurrent builds if not needed
- ✓ Use Pipeline Durability: Performance Optimized
- ✓ Store artifacts in external storage (S3, Nexus)
- ✓ Monitor & profile regularly



OPTIMIZATION CHECKLIST

- ✓ Right # of executors configured
- ✓ Parallel stages enabled
- ✓ Caching enabled
- ✓ Docker cache configured
- ✓ Workspaces cleaned
- ✓ Build retention configured
- ✓ Distributed agents used
- ✓ Monitoring in place



9. MONITOR & MEASURE



- Monitor key metrics
 - Build Duration
 - Queue Length
 - Executor Utilization
 - Agent Availability
 - Disk / Memory / CPU
- Tools: Prometheus, Grafana, Jenkins Performance Plugin, Node Exporter

SAMPLE DASHBOARD

BUILD DURATION



EXECUTOR USAGE



QUEUE LENGTH



VERIQTA

CONTINUOUS OPTIMIZATION = CONTINUOUS DELIVERY EXCELLENCE 100

VERIQTA

VERIQTA



16. JENKINS MONITORING



You can't improve what you don't monitor.
Monitor everything. Alert early. Fix faster.

VERIQTA

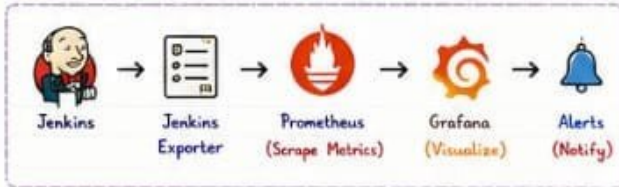
WHY MONITOR JENKINS?

- ✓ Ensure high availability and reliability
- ✓ Detect issues before users report them
- ✓ Optimize performance and resource usage
- ✓ Plan capacity and growth
- ✓ Improve security and compliance



VERIQTA

1. MONITORING ARCHITECTURE OVERVIEW



Data Flow:

Jenkins → Exporter → Prometheus → Grafana → Alertmanager → Notification

2. KEY MONITORING COMPONENTS

COMPONENT	ROLE	TOOLS
Metrics Collection	Collect metrics from Jenkins	Jenkins Prometheus Exporter
Metrics Storage	Store time-series metrics	Prometheus
Visualization	Dashboards & graphs	Grafana
Alerting	Send alerts on thresholds	Alertmanager
Log Aggregation	Collect & search logs	ELK / Loki
Health Checks	Check uptime & availability	Prometheus Blackbox Exporter

3. JENKINS METRICS (WHAT TO MONITOR)

- System Metrics: CPU, Memory, Disk, Load
- Jenkins Health: Uptime, Queue length, Executor usage
- Build Metrics: Build duration, Success/Failure rate
- Node/Agent Metrics: Online/Offline, Response time
- JVM Metrics: Heap usage, GC time, Threads, CPU
- Plugin Metrics: Plugin failures & compatibility
- Security Metrics: Failed login attempts, Errors



4. PROMETHEUS SETUP (JENKINS EXPORTER)



Add Jenkins Exporter Plugin

1. Manage Jenkins → Plugins
2. Install: "Prometheus Metrics"
3. Restart Jenkins

Access Metrics

<http://jenkins-server:8080/metrics>

(Text format for Prometheus)

Prometheus Config Example

scrape_configs:

```

- job_name: 'jenkins'
  metrics_path: '/prometheus'
  static_configs:
    - targets: ['jenkins-server:8080']
  
```

Scrape Interval: 15s (recommended)

5. GRAFANA DASHBOARD EXAMPLE



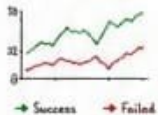
DASHBOARD PANELS

- ★ Jenkins Overall Health (Uptime)
- ★ Executor Usage (%)
- ★ Build Queue Length
- ★ Build Duration (Average)
- ★ Build Success/Failure Rate
- ★ JVM Heap Usage (%)
- ★ GC Time (ms)
- ★ Disk Usage (%)
- ★ Agent Status (Online/Offline)

Executor Usage



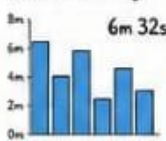
Builds (Last 24h)



JVM Heap Usage



Build Duration (avg)



6. ALERTS (ALERTMANAGER)



Common Alert Rules

- Jenkins Down
expr: up{job="jenkins"} == 0
for: 2m
- High Build Failure Rate
expr: rate(jenkins_builds_failed_total[15m]) > 0.2
- Executor Saturation
expr: jenkins_num_executors_in_use / jenkins_num_executors > 0.9
- High JVM Heap Usage
expr: jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes{area="heap"} > 0.8
- Disk Almost Full
expr: (node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" }) < 0.1

Notification Channels

- ✉ Email
- 📢 Slack
- 📠 Telegram
- 🏢 Microsoft Teams
- 📄 PagerDuty

Alert Flow

Prometheus
→ Alertmanager
→ Notification Channel

7. LOGS MANAGEMENT



Options

- ELK Stack (Elasticsearch + Logstash + Kibana)
- Grafana Loki
- CloudWatch / Splunk
- Filebeat / Fluentd

Log Sources

- Jenkins System Log
\$JENKINS_HOME/logs/jenkins.log
- Build Console Logs
- Access Logs
- Plugin Logs
- Agent Logs



Log Best Practices

- ✓ Centralize all logs
- ✓ Set retention policy
- ✓ Search & alert on Error/Exception
- ✓ Mask sensitive information



8. JVM MONITORING



Key JVM Metrics

- Heap Used (%)
- Non-Heap Used (%)
- GC Time (ms)
- GC Count
- Threads Count
- CPU Usage (%)

Useful JVM Tools

- VisualVM
- JConsole
- Prometheus JVM Exporter
- Grafana JVM Dashboard



9. DISK USAGE MONITORING



What to Monitor

- Jenkins Home (\$JENKINS_HOME)
- Build Workspaces
- Disk Space Usage (%)
- Inodes Usage
- Log Directory Size



Prometheus Disk Usage Example

```

- job_name: 'node-disk'
  static_configs:
    - targets: ['jenkins-server:9100']
  
```

(Node Exporter)

→ Alert if disk usage > 80% before your server cries!



MONITOR EVERYTHING. ALERT EARLY. STAY AHEAD.

100



VERIQTA



VERIQTA



VERIQTA





17. JENKINS PRODUCTION FAILURES

Real Production Case Files



Production will test everything you didn't.
Prepare. Monitor. Respond. Improve.

WHY LEARN FROM FAILURES?

- ✓ It helps you prevent future outages
- ✓ Reduces MTTR (Mean Time To Recovery)
- ✓ Builds strong, reliable CI/CD systems
- ✓ Turns incidents into engineering excellence



REAL PRODUCTION CASE FILES

1 PLUGIN UPDATE OUTAGE

What Happened

- A plugin update was installed without testing.

Impact

- Jenkins failed to start. All pipelines were down.

Root Cause

- Incompatible plugin version with Jenkins core.

Resolution

- Rollback plugin, restart Jenkins, test updates in a staging env.

2 DISK FULL

What Happened

- The Jenkins server disk filled up.

Impact

- Builds failed, no logs could be written.

Root Cause

- No log rotation, old builds not deleted.

Resolution

- Clean old data, enable log rotation, set build retention policy.

3 EXPIRED CERTIFICATE

What Happened

- Jenkins HTTPS certificate expired.

Impact

- Users could not access Jenkins.

Root Cause

- Certificate auto-renewal was not configured.

Resolution

- Renew certificate, enable auto-renewal with Let's Encrypt.

4 AGENT CRASH

What Happened

- Build agent went offline during job execution.

Impact

- Running jobs failed.

Root Cause

- Out of memory on agent machine.

Resolution

- Increase agent memory, configure auto-reconnect & monitoring.

5 DOCKER DAEMON FAILURE

What Happened

- Docker daemon crashed on the build agent.

Impact

- All Docker builds failed.

Root Cause

- Docker service unstable, resource exhaustion.

Resolution

- Restart Docker service, set resource limits, monitor daemon.

6 GITHUB WEBHOOK FAILURE

What Happened

- GitHub webhook stopped triggering builds.

Impact

- New code was not built automatically.

Root Cause

- Webhook URL changed / expired secret.

Resolution

- Reconfigure webhook, test delivery, monitor webhook health.

7 CREDENTIAL EXPIRATION

What Happened

- AWS credentials stored in Jenkins expired.

Impact

- Deployments to AWS failed.

Root Cause

- No alert before credential expiration.

Resolution

- Rotate credentials, set up alerts, use AWS IAM Roles if possible.

8 PLUGIN CONFLICT

What Happened

- Two plugins were incompatible after an update.

Impact

- Build errors and UI instability.

Root Cause

- Dependencies not checked before upgrade.

Resolution

- Check changelog, test in staging, update plugins carefully.

LESSONS LEARNED



- ✓ Always test updates in staging first.
- ✓ Monitor disk, memory, CPU, and logs.
- ✓ Set up alerts before problems happen.

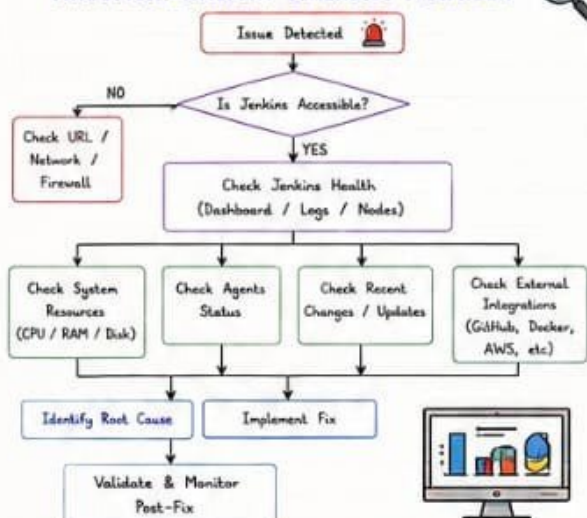
- ✓ Automate certificate renewal.
- ✓ Configure build retention & log rotation.
- ✓ Use stable plugin versions in production.

- ✓ Keep backups of Jenkins configuration.
- ✓ Use infrastructure as code for reproducibility.
- ✓ Document runbooks for common failures.

- ✓ Implement health checks for Jenkins & agents.
- ✓ Monitor external integrations (GitHub, Docker, AWS, etc.).
- ✓ Review and improve after every incident.



PRODUCTION INCIDENT DIAGNOSIS FLOWCHART



INCIDENT RESPONSE CHECKLIST

- DETECT** Alert triggered or issue reported.
- ACKNOWLEDGE** Acknowledge the incident immediately.
- ASSESS** Identify impact, scope, and severity.
- COMMUNICATE** Inform the team / stakeholders.
- INVESTIGATE** Check logs, metrics, and recent changes.
- RESOLVE** Apply fix or rollback if needed.
- VALIDATE** Verify the issue is resolved.
- POST-MORTEM** Document root cause, resolution, and actions to prevent recurrence.

USEFUL COMMANDS

```

Check Jenkins status
sudo systemctl status jenkins

View Jenkins logs
sudo journalctl -u jenkins -f

Check disk usage
df -h

Check memory
free -h

Check running processes
ps -ef | grep jenkins

Restart Jenkins
sudo systemctl restart jenkins

Check Docker status
sudo systemctl status docker
  
```

REMEMBER:

Fail fast. Learn faster.
Improve continuously.
That's how reliable systems are built.



FAILURES ARE TEMPORARY. LESSONS LAST FOREVER.

100





18. 50 JENKINS INTERVIEW QUESTIONS

Grouped Questions with Model Answers

VERIQTA

VERIQTA



HOW TO USE THIS PAGE

- ✓ Review questions by level.
- ✓ Practice explaining out loud.
- ✓ Understand the model answers.
- ✓ Adapt answers to your experience.
- ✓ Be confident in interviews!



1. BEGINNER LEVEL (1-10)



#	QUESTION	MODEL ANSWER (SUMMARY)
1	What is Jenkins?	→ Jenkins is an open-source automation server used for CI/CD.
2	What is the purpose of Jenkins?	→ Automate building, testing and deploying software.
3	What is a Job in Jenkins?	→ A Job is a task or set of steps executed by Jenkins.
4	What is a Build?	→ A build is the execution of a job.
5	What is a Plugin?	→ Plugins extend Jenkins functionality.
6	What is a Node (Agent)?	→ A node is a machine where Jenkins runs jobs.
7	What is the difference between Controller and Agent?	→ Controller manages jobs; Agents execute jobs.
8	What is a Pipeline?	→ A pipeline defines a CI/CD workflow as code.
9	What is a Freestyle project?	→ The classic job type configured via UI.
10	What is the Jenkins home directory?	→ The default is <code>~/jenkins</code> on Linux.

2. INTERMEDIATE LEVEL (11-20)



#	QUESTION	MODEL ANSWER (SUMMARY)
11	What is the difference between Freestyle and Pipeline?	→ Pipeline is code-based, versioned and more powerful.
12	How do you trigger a build?	→ Manually, on schedule, SCM change, webhook, etc.
13	What is SCM in Jenkins?	→ Source Code Management (e.g., Git, SVN).
14	What is a Jenkinsfile?	→ A file that defines a Pipeline as code.
15	What are Build Parameters?	→ They are inputs passed to a job at runtime.
16	What is the purpose of Post actions?	→ Run steps after build completion (success/failure).
17	How do you archive artifacts?	→ Using the "Archive the artifacts" option or <code>archiveArtifacts</code> step.
18	What is the difference between Global tool config and Environment?	→ Global tools are predefined; env vars are dynamic.
19	What is a Multibranch Pipeline?	→ A pipeline that runs for branches discovered in SCM.
20	How do you manage credentials in Jenkins?	→ Using Jenkins Credentials Store.

3. ADVANCED LEVEL (21-30)



#	QUESTION	MODEL ANSWER (SUMMARY)
21	What is shared Library in Jenkins?	→ Reuse Pipeline code across multiple projects.
22	What is the difference between Declarative and Scripted Pipeline?	→ Declarative is structured and simpler; Scripted is flexible and Groovy based.
23	How do you handle parallel execution?	→ Using <code>parallel</code> block in Pipeline.
24	What is the purpose of stages in Pipeline?	→ To break the workflow into logical steps.
25	What is the input step?	→ Pauses pipeline for user approval.
26	How do you handle secrets in Pipeline?	→ Use <code>withCredentials</code> and store in Jenkins credentials.
27	What is a custom plugin?	→ A plugin developed to extend Jenkins functionality.
28	How do you integrate Jenkins with Docker?	→ Use Docker Pipeline steps or Docker plugin.
29	What is a webhook?	→ An HTTP callback to trigger Jenkins builds.
30	How do you handle timeouts in Pipeline?	→ Using <code>timeout</code> step or build timeout settings.

4. PRODUCTION LEVEL (31-40)



#	QUESTION	MODEL ANSWER (SUMMARY)
31	How do you secure Jenkins?	→ Enable auth, HTTPS, RBAC, CSRF, least privilege.
32	How do you back up Jenkins?	→ Backup <code>\$JENKINS_HOME</code> , configs, plugins, secrets.
33	How do you monitor Jenkins?	→ Use Prometheus, Grafana, and Jenkins health metrics.
34	How do you handle master failure?	→ Use backups, HA setup or controller redundancy.
35	How do you scale Jenkins?	→ Use distributed agents and load balancing.
36	How do you clean old builds?	→ Use Build Discarder or log rotation policies.
37	How do you manage plugins in production?	→ Test updates in staging then promote.
38	What is the purpose of quiet period?	→ Wait time to batch multiple SCM triggers.
39	How do you handle disk space issues?	→ Clean workspaces, old builds, artifacts, and logs.
40	What is Jenkins Configuration as Code (JCasC)?	→ Manage Jenkins configuration using YAML files.

5. SCENARIO-BASED LEVEL (41-50)



#	SCENARIO QUESTION	MODEL ANSWER (SUMMARY)
41	Builds are failing due to 'No space left on device'. What will you do?	→ Clean old builds, artifacts, temp files; increase disk space; set log rotation.
42	Agents are frequently going offline. What steps will you take?	→ Check network, agent startup scripts, resource usage, logs; restart agent.
43	A plugin update broke Jenkins. How will you fix it?	→ Rollback to previous plugin version; test in staging before updating.
44	Builds are stuck in queue for a long time. What will you check?	→ Check executor availability, labels, node online status, builds running.
45	Credentials are not working in Pipeline. What will you do?	→ Verify credential ID, scope, binding in Pipeline, and scopes.
46	How will you deploy artifacts automatically after successful build?	→ Use <code>post { success [...] }</code> or deployment stage in Pipeline.
47	How will you restrict who can run or configure jobs?	→ Use Matrix-based security and Role-based access control plugin.
48	How will you integrate SonarQube with Jenkins?	→ Install plugin, configure server, use <code>withSonarQubeEnv</code> in Pipeline.
49	How will you ensure high availability for Jenkins?	→ Use HA setup, backup strategy, distributed architecture.
50	How will you troubleshoot a failed Pipeline build?	→ Check logs, console output, step by step, verify env, reproduce issue.

PRO TIPS



- ☆ Always explain with real examples.
- ☆ Use simple words and clear structure.
- ☆ Talk about how you solved problems, not just theory.
- ☆ Highlight tools: Jenkins, Docker, Git, Kubernetes, AWS, etc.
- ☆ Show automation mindset and reliability focus.



BONUS: MODEL ANSWER TIPS

- Keep answers short, clear and to the point.
- Use bullet points when possible.
- Add examples from your project experience.
- Show understanding of production environment.
- Focus on problem-solving approach.



KEY SKILLS INTERVIEWERS LOOK FOR

- ✓ Jenkins Pipeline (Declarative & Scripted)
- ✓ CI/CD Best Practices
- ✓ Troubleshooting & Debugging
- ✓ Security & Access Control
- ✓ Integration (Git, Docker, K8s, AWS)
- ✓ Monitoring & Performance
- ✓ Automation & Scalability
- ✓ Production Reliability Mindset



VERIQTA

VERIQTA



PRACTICE TODAY. PERFORM TOMORROW. AUTOMATE EVERYTHING.

100



VERIQTA



VERIQTA



19. JENKINS CHEAT SHEET



One Page. Everything You Need.

ABOUT JENKINS

- ✓ Open-source automation server
- ✓ CI/CD, automation, integration
- ✓ Extensible with 1800+ plugins



VERIQTA

1. USEFUL COMMANDS (LINUX) >_

```
# Start Jenkins
sudo systemctl start jenkins

# Stop Jenkins
sudo systemctl stop jenkins

# Restart Jenkins
sudo systemctl restart jenkins

# Check Status
sudo systemctl status jenkins

# Reload Config
sudo systemctl reload jenkins

# View Logs
sudo journalctl -u jenkins -f

# Backup Jenkins Home
tar -czvf jenkins-backup.tgz $JENKINS_HOME
```

2. IMPORTANT PORTS



PORT	PURPOSE
8080	Jenkins Web UI (Default)
50000	Jenkins Agent (JNLP)
443	HTTPS (If configured)
80	HTTP (If configured)
22	SSH (Agent / Server)
2376	Docker Daemon (TLS)
10250	Kubernetes Kubelet

3. MUST-HAVE PLUGINS



- ☆ Pipeline
- ☆ Git
- ☆ GitHub
- ☆ Docker Pipeline
- ☆ Kubernetes
- ☆ Credentials Binding
- ☆ Blue Ocean
- ☆ Matrix Authorization
- ☆ Email Extension
- ☆ Job DSL
- ☆ Prometheus Metrics
- ☆ AnsiColor

4. JENKINS DIRECTORY STRUCTURE



```
$JENKINS_HOME/
|-- jobs/           # All jobs
|-- builds/         # Build data
|-- workspace/      # Workspaces
|-- plugins/         # Installed plugins
|-- userContent/     # Static files
|-- config.xml       # Main config
|-- credentials.xml  # Credentials store
|-- nodes/           # Agent info
|-- secrets/         # Secret keys
|-- logs/            # Jenkins logs
|-- war/             # Jenkins WAR file
```

5. JENKINSFILE SYNTAX (QUICK REFERENCE)



Example Pipeline

```
pipeline {
  agent any
  environment {
    APP = "my-app"
  }
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/user/repo.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      steps {
        sh 'docker build -t my-app .'
        sh 'docker push my-app'
      }
    }
  }
}
```

Keywords

KEYWORD	DESCRIPTION
pipeline	Defines a Pipeline
agent	Where the pipeline runs
stages	Contains stages
stage	A phase in the pipeline
steps	List of steps
sh	Run shell command
bat	Run batch command (Windows)
environment	Define env variables
post	Actions after pipeline
when	Conditional execution
tools	Define tools (JDK, Maven)
parameters	User input parameters

Common Steps

- checkout scm
- git
- sh / bat
- echo
- sleep
- input
- archiveArtifacts
- junit
- docker.build()
- withCredentials()

6. GIT COMMANDS



COMMAND	USE
git clone <url>	Clone repository
git pull	Pull latest changes
git add .	Stage changes
git commit -m "msg"	Commit changes
git push origin <branch>	Push to remote
git checkout <branch>	Switch branch
git branch	List branches
git merge <branch>	Merge branch
git log --oneline	Compact commit log
git status	Check status

7. DOCKER COMMANDS



COMMAND	USE
docker build -t app .	Build image
docker images	List images
docker run -d -p 8080:80 app	Run container
docker ps	List running containers
docker ps -a	List all containers
docker stop <id>	Stop container
docker rm <id>	Remove container
docker rmi <image>	Remove image
docker login	Login to registry
docker push <image>	Push image
docker logs <id>	View logs
docker exec -it <id> sh	Access container shell
docker system prune -a	Clean unused data

8. KUBERNETES COMMANDS



COMMAND	USE
kubectl get pods	List pods
kubectl get nodes	List nodes
kubectl get svc	List services
kubectl get all	All resources
kubectl describe pod <pod>	Describe pod
kubectl logs <pod>	View pod logs
kubectl exec -it <pod> -- /bin/sh	Access pod shell
kubectl apply -f file.yaml	Apply manifest
kubectl delete -f file.yaml	Delete manifest
kubectl rollout status deploy/<name>	Check rollout status
kubectl config get-contexts	List contexts
kubectl top pods	Pod resource usage
kubectl drain <node> --ignore-daemonsets	Drain node

9. SHORTCUTS (JENKINS UI)



SHORTCUT	ACTION
/	Open search
ctrl + k	Open global search
ctrl + r	Reload page
ctrl + f	Find on page
g then j	Go to Jenkins dashboard
g then n	Open nodes page
g then c	Open credentials
g then p	Open plugins
g then u	User management
esc	Close dialogs / modals
? (help)	Open help

10. INTERVIEW TIPS



- ✓ Understand CI/CD concepts deeply
- ✓ Know Jenkins architecture & components
- ✓ Be clear on Pipelines (Declarative & Scripted)
- ✓ Hands-on with real projects
- ✓ Know how to troubleshoot failures
- ✓ Understand security best practices
- ✓ Be ready for scenario-based questions
- ✓ Practice common Jenkins commands
- ✓ Stay updated with latest plugins & features
- ✓ Think like a DevOps engineer



VERIQTA

11. COMMON INTERVIEW TOPICS

- ☆ Jenkins architecture & components
- ☆ Pipeline syntax (Declarative vs Scripted)
- ☆ Jenkinsfile & Pipeline Stages
- ☆ Agents, Nodes & Executors
- ☆ Plugins & Their usage
- ☆ Credentials & Security
- ☆ Integration with Git, Docker, Kubernetes
- ☆ Build triggers (Poll, Webhook, SCM)
- ☆ Troubleshooting real-world issues
- ☆ Monitoring Jenkins
- ☆ Backup, Restore & Maintenance
- ☆ Best Practices for Production



12. INTERVIEW MODEL ANSWER EXAMPLE



Q: What is the difference between Freestyle and Pipeline job?

Answer:

Freestyle job is the traditional way of creating jobs in Jenkins with UI-based configuration and limited scalability.

Pipeline job uses Jenkinsfile (code) to define the entire CI/CD workflow. It is version-controlled, repeatable, scalable, and supports complex workflows with stages, parallel execution, and integrations.

Key Points:

→ Freestyle: UI-based, hard to maintain

→ Pipeline: Code-based, versioned, scalable, reusable



PRACTICE. BUILD. AUTOMATE. REPEAT. THAT'S THE JENKINS WAY! 100

VERIQTA





20. PRODUCTION READINESS CHECKLIST

Ship It. Run It. Trust It.

PRODUCTION PRINCIPLES



- ☒ Security first
- ☒ Automate everything
- ☒ Reliability always
- ☒ Test before you trust
- ☒ Observability everywhere
- ☒ Plan for failure, design for recovery

1. SECURE JENKINS

- ☒ Enable HTTPS with valid certificate
- ☒ Disable HTTP
- ☒ Enable Matrix Authorization (RBAC)
- ☒ Use least privilege for all users
- ☒ Protect against CSRF (enabled)
- ☒ Use strong password policy
- ☒ Enable 2FA for admins
- ☒ Restrict CLI access (SSH whitelist)
- ☒ Disable anonymous access
- ☒ Regular security audit & log review



2. BACKUP STRATEGY

Backup What?

- \$JENKINS_HOME (configs, jobs, plugins)
- Secrets & Credentials
- Plugins list
- Job DSL / Jenkinsfiles
- Custom scripts & tools
- System settings

How Often?

- Daily incremental
- Weekly full backup

Where?

- Offsite / Encrypted storage (S3, GCS, etc.)

3. HA CONSIDERATIONS

- ☒ Run Jenkins behind Load Balancer
- ☒ Multiple Controller (Active/Passive)
- ☒ Shared storage or replicated \$JENKINS_HOME
- ☒ Use External Database (e.g., PostgreSQL)
- ☒ Use Shared Agent Pool
- ☒ Enable Session Affinity (sticky sessions)
- ☒ Monitor controller health
- ☒ Test failover regularly

Tip: If Jenkins is down, your delivery is down.

4. PLUGIN MANAGEMENT

- ☒ Install only required plugins
- ☒ Keep plugins up-to-date
- ☒ Check compatibility before update
- ☒ Test in staging environment
- ☒ Pin plugin versions
- ☒ Review plugins periodically
- ☒ Remove unused plugins
- ☒ Use plugin catalog / lock file
- ☒ Backup before plugin update

Outdated plugins = Security Risk

5. MONITORING & OBSERVABILITY

Monitor Everything

- ☒ Jenkins Health (Uptime, Queue, Executors)
- ☒ Builds (Success, Failure, Duration)
- ☒ System Resources (CPU, RAM, Disk)
- ☒ Agents (Online/Offline, Usage)
- ☒ Log errors & exceptions
- ☒ JVM Metrics (Heap, GC, Threads)

Tools

- Prometheus + Grafana
- ELK / Loki for logs
- Alertmanager for alerts



6. DISASTER RECOVERY

Be Ready When Things Break

- ☒ Document DR plan
- ☒ Automate backup & restore
- ☒ Store backups offsite
- ☒ Test restore regularly
- ☒ Infrastructure as Code (Terraform/CloudFormation)
- ☒ Rebuild controllers from code
- ☒ Have runbooks for common failures
- ☒ RTO / RPO defined & tested

RTO: Recovery Time Objective
RPO: Recovery Point Objective

7. PERFORMANCE TUNING

Optimize for Speed

- ☒ Use more executors (but not too many)
- ☒ Enable Pipeline & Docker layer caching
- ☒ Clean up workspaces regularly
- ☒ Limit concurrent builds if needed
- ☒ Use lightweight agents
- ☒ Store artifacts in external storage (S3)
- ☒ Tune JVM (Heap size, GC, Threads)
- ☒ Remove old builds & artifacts

Check Regularly:

- Queue length
- Build duration
- Agent utilization



8. UPGRADE STRATEGY

Upgrade Smart, Not Hard

- ☒ Read changelog & plugin compatibility
- ☒ Backup everything
- ☒ Upgrade in staging first
- ☒ Run smoke tests
- ☒ Upgrade during low traffic window
- ☒ Rollback plan ready
- ☒ Validate integrations after upgrade
- ☒ Documentation update



9. FINAL DEVOPS CHECKLIST (FOR PRODUCTION)

A. SECURITY

- ☒ HTTPS enabled
- ☒ RBAC configured
- ☒ Least privilege applied
- ☒ Credentials encrypted
- ☒ Audit logs enabled
- ☒ Agents secured
- ☒ Network access restricted

B. RELIABILITY

- ☒ HA configured
- ☒ Backups automated
- ☒ DR plan tested
- ☒ Monitoring in place
- ☒ Alerts configured
- ☒ Controllers healthy
- ☒ Agents healthy

C. MAINTAINABILITY

- ☒ Documentation up-to-date
- ☒ Jobs stored in Git
- ☒ Pipeline as Code used
- ☒ Reusable Shared Libraries
- ☒ Code review for changes
- ☒ Standard naming conventions
- ☒ Clean & organized structure

D. PERFORMANCE

- ☒ Executors tuned
- ☒ Resources monitored
- ☒ Artifacts cleaned
- ☒ Workspaces cleaned
- ☒ Caching enabled
- ☒ Build times optimized
- ☒ Infrastructure scalable

E. DELIVERY EXCELLENCE

- ☆ Automate pipeline
- ☆ Monitor or you will miss it.
- ☆ Tests automated
- ☆ Deployments automated
- ☆ Rollback automated
- ☆ Notifications enabled
- ☆ Improve it or you will fall behind.

USEFUL ENVIRONMENT VARIABLES

- \$JENKINS_HOME → Jenkins home directory
- \$WORKSPACE → Current workspace
- \$BUILD_NUMBER → Build number
- \$BUILD_URL → Build URL
- \$JOB_NAME → Job name
- \$NODE_NAME → Agent/Node name
- \$BUILD_ID → Unique build ID



COMMON \$JENKINS_HOME DIRECTORIES

- jobs/ → Job configurations
- config.xml → Main Jenkins configuration
- credentials.xml → Credentials store
- plugins/ → Installed plugins
- workspace/ → Workspaces
- userContent/ → Static files, scripts
- logs/ → Jenkins logs
- secrets/ (if present) → Secrets (older versions)



PRODUCTION GOLDEN RULES

- ☆ Automate or eliminate it.
- ☆ Monitor or you will miss it.
- ☆ Secure it or someone will break it.
- ☆ Backup it or you will lose it.
- ☆ Document it or you will forget it.
- ☆ Test it or you can't trust it.
- ☆ Improve it or you will fall behind.



A WELL-RUN JENKINS IS THE HEART OF A HIGH-PERFORMING DEVOPS CULTURE. 100

